IEEE JOURNAL ON
SELECTED AREAS IN COMMUNICATIONS

IEEE Xplore Digital Library
http://ieeexplore.ieee.org/Xplore

Perception Task Offloading with Collaborative Computation for Autonomous Driving

Zhu Xiao, *Senior Member, IEEE*, Jinmei Shu, Hongbo Jiang, *Senior Member, IEEE*, Geyong Min, Hongyang Chen, *Senior Member, IEEE*, and Zhu Han, *Fellow, IEEE*

***Abstract*—Autonomous driving has so far received numerous attention from academia and industry. However, the inevitable occlusion is a great menace to safety and reliable driving. Existing works have primarily focused on improving the perception ability of a single autonomous vehicle (AV), but the safety problem brought by occlusions remains unanswered. In this paper, we propose a multi-tier perception task offloading framework with a collaborative computing approach, where an AV is able to achieve a comprehensive perception of the concerned region-of-interest (RoI) by leveraging collaborative computation with nearby AVs and road side units (RSUs). Besides, the collaborative computation provides offloading service for computationally intensive tasks so as to reduce processing delay. Specifically, we formulate a joint problem of perception task assignment, offloading and resource allocation, by fully considering the AV's mobility, task dependency, and delay requirement. The collaborative offloading is modeled as a mixed-integer nonlinear programming (MINLP) problem. We design a two-layer binary intelligent firefly (TL-BIFA) algorithm to solve MINLP, with the goal of minimizing execution delay. The proposed TL-BIFA synthesizes the advantages of heuristic methods and deterministic methods. Through extensive simulations, the proposed collaborative offloading approach and the TL-BIFA show superiority in enhancing the autonomous driving system's safety, efficiency and resource utilization.**

***Index Terms*—Perception task offloading, collaborative computation, autonomous driving, vehicle mobility.**

I. Introduction

THE boom of *autonomous driving* in recent years has expanded itself to the territories beyond technological achievement [1], [2]. Equipped with cutting-edge sensors such as ultrasonic radar, LiDAR and stereo cameras, autonomous vehicle (AVs) are provided with varied fields of view [3]. Using a combination of artificial intelligence processing units, AVs are able to make decisions in complex driving situations without human input [4]. Furthermore, the autonomous driving with the Internet-of-Vehicles (IoV) enables information sharing among connected vehicles, promising a congestion-free, non-accident and energy-saving transportation system.

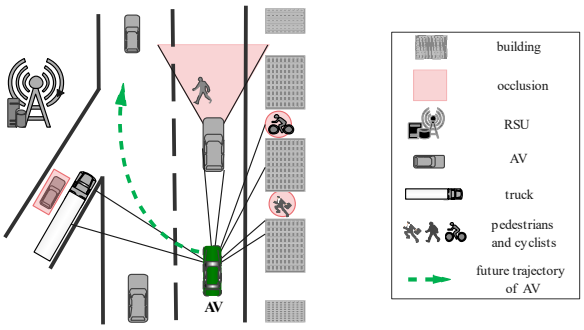

Fig. 1. Occlusions in autonomous driving. For the AV (marked 'green'), its perceivable range is limited due to various occlusions, i.e., the speeding vehicle blinded by the truck, the pedestrians blocked by the vehicle in front, the cyclists and the walker between the buildings. Car collision or traffic accident would happen if the AV could not acquire real-time knowledge on these occlusions.

Although the substantial success of autonomous driving is on the way, the limited perceivable range of a single AV tends to degrade the automated driving performance. This is largely because of the unavoidable occlusions during driving. Even if an AV is equipped with a 360° camera and a high-end LiDAR, it is unable to perceive the occlusions caused by dynamic or static obstacles (e.g., nearby AVs and buildings). As shown in Fig. 1, the AV (marked 'green') needs to perceive the surroundings to realize reliable planning and control before any actions are taken. However, the speeding vehicle blinded by the truck, together with the pedestrians and cyclists, are beyond the AV's perceivable range, and consequently causes occlusions. If the AV intends lane-changing without acquiring knowledge of those occlusions, traffic accidents or car collision may well happen.

As a promising solution, leveraging collaborative computation empowers the AV to regain the visual information of occlusions. By feat of the collaborative computation, the AV is able to establish collaboration with road-side units (RSUs) and nearby AVs in the automotive environment, whose sensors

Manuscript received XX XX 2022.

This work was supported in part by the National Natural Science Foundation of China under Grants 62272152, 62271452 and U20A20181; in part by the Key R&D Project of Hunan Province of China under Grant 2022GK2020; in part by Hunan Natural Science Foundation of China under Grant 2022JJ30171, and in part by the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ) under Grants GML-KF-22-22 and GML-KF-22-23; in part by NSF CNS-2107216, CNS-2128368 and CMMI-2222810; in part by Shenzhen Science and Technology Program under Grant JCYJ20220530160408019; in part by the Funding Projects of Zhejiang Lab under Grants 2021LC0AB05 and 2022PI0AC01. (*Corresponding author: H. Jiang.)*

Z. Xiao, J. Shu and H. Jiang are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, 410082, China, and also with Shenzhen Research Institute of Hunan University, Shenzhen, 518055 China. (email: {zhxiao, jinmeishu, hongbojiang}@hnu.edu.cn).

G. Min is with the Department of Computer Science, University of Exeter, Exeter, UK. (email: g.min@exeter.ac.uk).

H. Chen is with the Research Center for Graph Computing, Zhejiang Lab, Hangzhou 311100, China. (email: hongyang@zhejianglab.com).

Z. Han is with is with the Department of Electrical and Computer Engineering at the University of Houston, Houston, TX 77004 USA, and also with the Department of Computer Science and Engineering, Kyung Hee University, Seoul, South Korea, 446-701. (e-mail: hanzhu22@gmail.com).

Digital Object Identifier: 10.1109/JSAC.2022.3227027

cover the occlusions. For example, in Fig. 1, the AV (the 'green' vehicle) can be informed of a potential car crash by collaborating with the RSU through vehicle-to-infrastructure (V2I) link. As for the vision blocked by the vehicle in front, it can be recovered by communicating with the vehicle ahead via vehicle-to-vehicle (V2V) links. As such, co-perceiving the surroundings with nearby AVs and RSUs not only benefits a single AV, but also forms a global information network, safeguarding all the AVs and pedestrians. However, collaborative computation alone does not promise the absolute safety for the autonomous driving, since traffic accidents take place if the perceived data is not analysed in a timely manner. Note that the process of perception tasks often requires computation-intensive machine learning (ML) models, which makes it hard for the resource-constrained AVs to react to the traffic condition in real-time. To tackle this issue, perception task offloading enables outsourcing the collaborative computation workload to RSUs. In this way, perception task offloading with collaborative computation provides the AV with enhanced and prompt awareness of its surroundings, and consequently improving driving reliability and road safety.

Existing works have investigated the collaborative computation and perception task offloading. For instance, the authors in [5] and [6] focused on extending the AV's field of view by means of collaborative computation. In detail, Sridhar and Eskandarian [5] turned every nearby AV into a moving platform via V2V communication. Guizar *et al.* [6] adopted vehicle-to-everything (V2X) communication to facilitate collaborative perception. A successful collaborative perception demands real-time processing. Regarding this, the authors in [7] and [8] applied task offloading strategy to autonomous driving systems. Instead of merely focusing on optimizing the assignments of computation tasks, the authors in [9] and [10] proposed computing resource allocation strategies to boost the performance of task offloading.

Despite the efforts, several challenging issues remain unresolved in the aforementioned works. *i*) None of these works took AVs' mobility into consideration when performing collaborative perception tasks. During the movement of the AV, its accessible AVs and RSUs change, as well the occlusions. The information of occlusions will lost if the AV moves out of the communication range in the collaborative computation, or if the occlusion is no longer under the sensing range of the co-perceivers (e.g., the RSUs and nearby AVs). *ii*) Existing studies neglect the inherent dependency of perception tasks in collaborative computation, which is likely to result in infeasible solutions. For example, according to recent designs of automated autonomous driving systems from both academic [8], [11], [12] and industry [13], [14], in the perception stage, object tracking can start only when object detection is finished. Task failure will turn out if object tracking is scheduled to be executed before detection. *iii*) Efficient task offloading requires a well-balanced resource allocation strategy. Allotting more resources to one task helps to accelerate its execution process, while the performance of other offloaded tasks will be degraded, thereby bringing higher delay to complete collaborative computation. In other words, an inappropriate resource allocation strategy leads to prolonged computation delay, degrading offloading performance and hindering the safety of autonomous driving.

Motivated by the above-mentioned challenges, we propose a new perception task offloading framework with collaborative computation to enhance the reliability of autonomous driving. Specifically, to resolve the mobility issue, the AV's movement is modeled as a three-phase process. The candidate co-perceivers for collaborative computation are identified according to the position features during each phase. Furthermore, task offloading and resource allocation are enabled for AVs with collaborative computation to accelerate task execution. To this end, we formulate a joint problem of perception task assignment, offloading and resource allocation, via fully considering the AV's mobility, the task dependency and the delay requirement. The offloading decisions are made under the constraints of vehicle mobility and task dependency. Besides, the resource allocation issue is tackled for task offloading, so that computing resources are utilized effectively and the completion delay of perception task is further reduced.

The main contributions of this paper are summarized as follows.

- In the multi-tier perception task offloading framework with collaborative computation, we propose to generalize the AV's movement as a three phase-process. To specify, we define the AV's concerned area as a region of interest (RoI) box, and each phase is characterized by the relative position of RoI and RSUs. Collaborative computation is carried out based on the AV's current phase.
- An original and efficient perception task offloading strategy is designed to minimize perception delay, under the constraint of task dependency. A perception task contains three parts, namely localization, detection and tracking. Object detection and tracking tasks are scheduled in a sequential manner, while localization is performed in parallel with these two tasks. In so doing, the dependency of perception task is met. Besides, the uploading method of background subtraction is utilized for perception task offloading, so as to reduce the uploaded perceived data.
- A resource allocation mechanism is proposed to improve the performance of task offloading. To specify, we formulate a joint problem of perception task assignment, offloading and resource allocation, with the goal of minimizing the task completion delay. The formulated problem is a mixed-integer nonlinear programming (MINLP) problem. A two-layer binary intelligent firefly algorithm (TL-BIFA) is proposed to solve this problem. In the outer layer, a binary intelligent firefly algorithm (BIFA) is designed to search for the optimal offloading solution. With the integer variables fixed, the original problem is decomposed into a nonlinear programming (NLP) problem. Based on the KKT conditions, the resource allocation strategy is achieved, and hence the NLP problem is settled.
- Experimental results show that our proposed approach outperforms others in terms of fast convergence, high resource utilization and complete sensory information. With 400 *TOPS* of vehicle's computing power, the collaborative computation scheme can reach a minimal perception

delay of 0.14 *ms* per video frame, which is far less than the delay upper bound of 3 *ms* when the AV is driving at a speed of 50 km/h. The resource utilization and the information completion are up to 100%, which verifies the effectiveness and reliability of the proposed approach.

The remainder of this paper is organized as follows. In Section II, we present an overview of the related works. Section III presents the system model. Problem formulation and the corresponding solution are presented in Section IV and Section V, respectively. Section VI provides the evaluation and result discussion. Finally, Section VII concludes the paper.

II. Related Works

There have been a number of works focusing on the occlusion problem in automotive environments. Naumann *et al.* [15] predicted the safety for an AV to pass through a potential conflict area with occluded vehicles. Schratter *et al.* [16] combined an offline approach with emergency braking system to obtain less conservative driving behavior for a pedestrian collision. Zhang *et al.* [17] predicted the appearance probability of phantom objects in occlusion scenarios. Apart from the prediction-related approaches, efforts have been devoted to collaborative computation for addressing occlusions in automated environments. Collaborative computation refers to collaborating with nearby AVs and/or RSUs to perceive the surroundings. Ghorai *et al.* [18] utilized the on-board sensory perception capabilities and information obtained through V2X connections to enhance driving safety. Hui *et al.* [19] proposed a digital twins based scheme to facilitate the collaborative and distributed autonomous driving.

Perception tasks are computation-intensive, and the delay requirement is as stringent as a few milliseconds [20]. To shorten the computation delay, the task offloading methodology has been introduced to autonomous driving systems. Liu *et al.* [21] developed an intelligent vehicle scheduling and offloading algorithm to reduce offloading latency. Qi *et al.* [22] exploited the sensing ability of AVs for real-time data collection, vehicular edge computing was adopted to improve the computing capability, and to reduce the transmission of massive raw environmental data. An edge learning-based offloading framework for autonomous driving was proposed by Yang *et al.* [23], where the deep learning tasks can be offloaded to the edge server to improve the inference accuracy while meeting the latency constraint. Yet, the perceived data generated by various sensors are up to 2 GB/s, these data requires processing with extremely tight latency constraints [24]. To reduce the high transmission delay incurred by uploading of an enormous amount of raw sensing data, Qi *et al.* [7] developed a task offloading mechanism, where the vehicle can either use the conventional offloading method to upload the raw perceived data, or it can adopt the instruction transmission method. The instruction transmission allowed the AV to transmit computation instructions instead of raw sensory data, the uploading data are significantly decreased.

To ensure the offloading performance, many researches have been concentrated on working out reasonable resource allocation strategies. Note that RSUs and vehicles are both under the constraint of computing power. If excessive computing resources are squandered on one task, the others' computing delay is prone to be lengthened, and the overall collaborative computation process is enlarged. To address this issue, Du *et al.* [25] devised a continuous relaxation and Lagrangian dual decomposition based iterative algorithm for joint radio resource and power allocation. Feng *et al.* [26] focused on the optimization of task offloading and resource allocation, the original optimization problem was transformed into an equivalent weighted-sum problem. A low-complexity greedy based efficient searching algorithm was proposed to obtain offloading strategies. Liu *et al.* [27] proposed a game-based distributed computation scheme, where the users compete for the edge cloud's finite computation resources via a pricing approach. Huang *et al.* [28] aimed at maximizing the offloaded computation data by task partition, transmit power and offloading time allocation, without consideration of computing resource allocation. Dai *et al.* [29] proposed computation offloading and resource allocation optimization for augmented vehicular reality algorithm with hybrid sensing data fusion of cooperative perception. The optimal power allocation was carried out based on convex optimization theory.

For collaborative computation, there exists a particular order for tasks to be performed. Specifically, object detection is the preceding task of object tracking, the later cannot start until the former is completed. Cui *et al.* [8] considered task dependency when offloading automotive tasks, and proposed a novel approach to offload computationally intensive autonomous driving services. Liu *et al.* [30] developed an efficient task scheduling algorithm to prioritize multiple tasks so as to guarantee the completion time constraints. Collaborative computation also requires tackling the mobility of the AV. The movement of the AV brings dynamic to the occlusions and its accessible objects, and consequently leads to the variation of its concerned area and intermittent connectivity. In [31], Mao *et al.* presented a data-driven capacity planning framework, regarding the deployment of stationary and mobile fog nodes. They aimed to minimize the installation and operational costs, while considering the spatio-temporal variation in both demand and supply. In [32], Liu *et al.* proposed a task offloading scheme by exploiting multi-hop vehicle computation resources based on mobility analysis of vehicles. An optimization problem was formulated to minimize the weighted sum of execution time and computation costs. Nevertheless, these approaches concentrated only on the computation part, without addressing the AV's perceptive limitation issue.

Summarizing, collaborative computation is a promising solution to expand the AV's field of view, yet it demands the real time processing of the computation-intensive tasks. On top of that, the limitation on AV's perceptive range was essentially unsolved in the existing works. Perception task offloading accelerates the task execution process, while on the premise that vehicle mobility, task dependency and resource allocation are considered in a joint way. Inspired by this, we propose the perception task offloading with collaborative computation for autonomous driving. The collaborative computation is carried out based on the movement of the AV, which is modeled as a three-phase process (see Section III-C). During the

task offloading process, the uploading method of background subtraction (see Section III-D1) is exploited to reduce the transmission delay. The scheduling of perception tasks follows the constraint of task dependency (see Section III-D2), and the computing resources are wisely allocated to minimize the completion delay (see Section IV).

III. Syetem Model

In this section, we present a generic autonomous driving scenario where multiple AVs move with several equally spaced RSUs along the road. During AV's movement, it is required the environmental perception for the safety driving. When occlusions occur, the AV can demand its co-perceivers, i.e., the nearby AVs and RSUs, to help perceiving its RoI. The perception tasks can be processed by vehicles themselves or offloaded to the serving RSUs. A central unit (CU), e.g., a base station (BS), is in charge of action executions in the multi-tier collaborative computation [7]. The CU collects the required information in the autonomous driving system, conducts the task assignment, makes offloading and resource allocation decisions and informs RSUs and vehicles of the decisions. The data transmission and information sharing are supported by V2V and V2I communications, through which the Ultra Reliable Low Latency Communication (URLCC) can achieve low latency with less than 1*ms* [33].

The system model of the perception task offloading with collaborative computation is described in the following subsections. Table I presents the main notations and definitions in this paper.

Remarks. RSU is willing to help the AV in the computation of perceptive tasks, since it is deployed as a stable participant in the autonomous driving system to improve the system's safety level. Besides, AVs' participating in collaborative computation is not only about helping the single AV, but also about the safety of its own and the whole road. Beyond that, the intention of perceiving public areas in the automotive environment makes collaborative computation exclusive of the privacy concern.

A. Perception Task in Autonomous Driving

The autonomous driving technology includes two stages, i.e., perception and decision [12]. Fig. 2 illustrates the perception tasks and computing pipeline of a highly automated autonomous driving system. The perception stage contains environmental data collection, vehicle localization, object detection and object tracking. In particular, the output of object detection is the input of object tracking, infeasible computation offloading solution is caused if the task dependency is neglected. Integrating both perceptive information and current driving mode (e.g., driver's interventions), the autonomous driving system is able to perform motion planning [34], which includes action prediction and obstacle avoidance. In this paper, we focus on the perception stage, based on which the AV acts to the automotive environment. The performance of perception tasks determines the reliability of autonomous driving.

TABLE I
Notations and Definitions

Notations	Definitions
f_{v_i}	Computing resource of Vehicle v_i
f_{u_j}	Computing resource of RSU u_j
$\mathcal{R}_{RoI}$	RoI of v_0
$\mathcal{R}_0$	Perceivable area of v_0 in $\mathcal{R}_{RoI}$
$\mathcal{R}_f$	Occluded area in front of v_0 in $\mathcal{R}_{RoI}$
$\mathcal{R}_b$	Occluded area behind v_0 in $\mathcal{R}_{RoI}$
$\mathcal{C}_{RoI}$	Perception task of v_0 in $\mathcal{R}_{RoI}$
$\mathcal{C}_0$	Perception task of v_0 in $\mathcal{R}_0$
$\mathcal{C}_f$	Perception task of v_0 in $\mathcal{R}_f$
$\mathcal{C}_b$	Perception task of v_0 in $\mathcal{R}_b$
$\mathcal{V}_f$	Set of AVs ahead of v_0 in $\mathcal{R}_{RoI}$
$\mathcal{V}_b$	Set of AVs behind v_0 in $\mathcal{R}_{RoI}$
ρ	Phase indicator
x_i^*	Task offloading indicator
$\mu_{v_i}^*$	Resource allocation indicator of vehicle v_i
$\mu_{u_j}^*$	Resource allocation indicator of RSU u_j
Ω	Set of AVs and RSUs that are assigned collaborative perception tasks
Ω_V	Set of AVs that are assigned collaborative perception tasks
Ω_U	Set of RSUs that are assigned collaborative perception tasks
$\mathcal{X}$	Task offloading decision for AVs in Ω_V
D_{v_i}	Completion delay of v_i in Ω_V
D_{u_j}	Completion delay of u_j in Ω_U
μ_{v_i}	Overall fraction of allocated resources to v_i in Ω_V
μ_{u_j}	Overall fraction of allocated resources to u_j in Ω_U

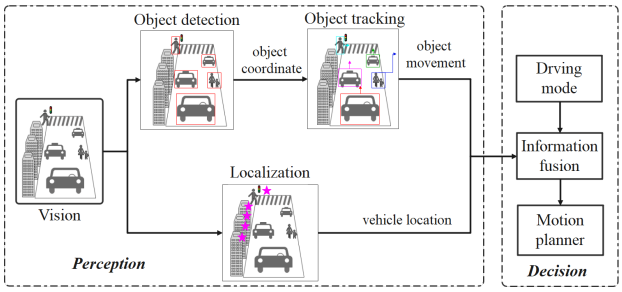

Fig. 2. Perception task in autonomous driving. At the *perception* stage, the captured video is streamed in parallel to *i*) object detector and tracker, to recognize and trace objects; *ii*) localization engine to locate the AV. At the *decision* stage, object movements, vehicle location and driving information are fused and transferred to motion planner for actions.

The *task dependency* can be explained as follows. First, AV's visual sensors capture the video of the automotive environment and transfer it into the object detector and the localization engine concurrently. Then, the detected objects are passed to the object tracker to follow the objects' movement. The outputs of AV's localizer and tracker, i.e., AV's localization and the tracked objects, are projected into the same 3D coordinate space by the fusion machine. After that, the motion planner makes operational decision according to the results of information fusion. Finally, the motion planner is invoked once the AV deviates from its originally planned route, wherein the information is delivered to motion planner for a more informed operational plan.

Remarks. Existing studies devoted to task offloading for autonomous driving often schedule these tasks in a sequential way, while indeed localization task can be executed at the same time with detection and tracking. Besides, object detection should be processed before tracking to obtain object movement. On this basis, we schedule task offloading based on the inherent dependency of perception tasks.

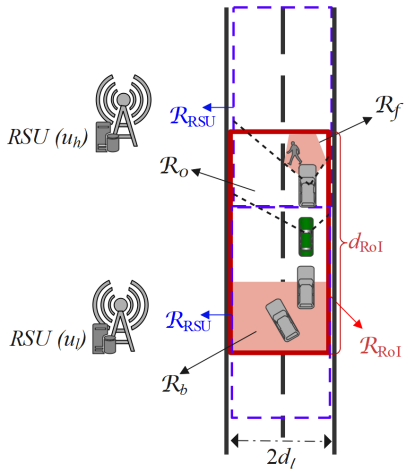

Fig. 3. Perception task offloading with collaborative computation. $\mathcal{R}_0$ is the perceivable region of AV v_0 (e.g., the green vehicle). $\mathcal{R}_f$ and $\mathcal{R}_b$ are the occlusions in the concerned area $\mathcal{R}_{RoI}$ (the red solid rectangle). By collaborating with AVs and RSUs, the AV acquires a comprehensive understanding for $\mathcal{R}_{RoI}$. Offloading perception tasks to RSUs further accelerates the understanding process.

B. Perception Task with Collaborative Computation

Recall the first paragraph of Section III, which describes the system model of perception task offloading with collaborative computation. For the sake of analytical simplicity, we consider a generalized scenario in the autonomous driving environment, namely, as illustrated in Fig. 3, a one-directional road comprising two lanes, and the width of each lane is d_l. RSUs equipped with visual sensors are localized equidistantly along the road. The computing power of an RSU is f_{u_j}. The sensing support of an RSU is approximated by a rectangular with a length of d_r and a width of $2d_l$, which is denoted by $\mathcal{R}_{RSU}$ (see the blue dashed rectangle in Fig. 3) [35].

There are N AVs driving on the road, each AV is equipped with a front camera. Let $\mathcal{V} = \{v_0, \ldots, v_i, \ldots, v_N\}$ denote the group of AVs. The computing resource of AV v_i is denoted by $f_{v_i}, v_i \in \mathcal{V}$. The field of view (FoV) of visual sensor is modeled as an isosceles triangle and can be described by $F_{v_i} = \{\alpha, \varphi\}$ [36], where α is the FoV vertex angle, denoting the aperture of the camera. The parameter φ is the complementary angle to $\alpha/2$, which defines the pose of the camera.

As a running example, let say AV v_0 (see the green vehicle in Fig. 3) needs to perceive the surrounding environment. Following [37], we describe the concerned area as a region of interest (RoI) box $\mathcal{R}_{RoI}$, with a length of d_{RoI} and a width of $2d_l$. Note that using road classification algorithms such as [38], the RoI can be identified in a timely manner according to ever-changing surroundings in practical driving situations. Owing to the occlusions caused by buildings and other vehicles, not all the areas in RoI can be perceived by v_0. As shown in Fig. 3, the occluded zones of v_0 include occlusions in front and blind spots behind. $\mathcal{R}_{RoI}$ is further divided into three parts: $\mathcal{R}_0$, $\mathcal{R}_f$ and $\mathcal{R}_b$. $\mathcal{R}_0$ is the area within v_0's sight of view, $\mathcal{R}_f$ and $\mathcal{R}_b$ are the occlusions in front of and behind v_0, respectively. Hence, we have

$$\mathcal{R}_0 + \mathcal{R}_f + \mathcal{R}_b = \mathcal{R}_{RoI}. \tag{1}$$

Potential threat exists in $\mathcal{R}_f$ and $\mathcal{R}_b$ since there might be pedestrians or overtaking cars.

For seeking solutions, we leverage collaborative computation to upgrade the safety level of autonomous driving, where the perception tasks are offloaded and handled by nearby AVs and RSUs collaboratively. In this way, the perception tasks are computed, and the computing results are sent back to v_0 through V2V or V2I links. In so doing, the occlusions of v_0 are eliminated, since v_0 is able to acquire a complete knowledge on its concerned area. Corresponding to (1), RoI perception task $\mathcal{C}_{RoI}$ is divided into three parts, i.e., $\mathcal{C}_0$, $\mathcal{C}_f$ and $\mathcal{C}_b$, which denote the perception task in $\mathcal{R}_0$, $\mathcal{R}_f$ and $\mathcal{R}_b$, respectively. The reason we partition task $\mathcal{C}_0$ is for the convenience of finding co-perceivers to assign collaborative computation tasks among, which is based on the knowledge of where the occlusions are. Accordingly, we obtain

$$\mathcal{C}_0 + \mathcal{C}_f + \mathcal{C}_b = \mathcal{C}_{RoI}. \tag{2}$$

Remarks. The generalized model (see in Fig. 3) can be applied into various automotive scenarios. The AV broadcasts perception request of its RoI when meeting occlusions. Knowing the request, the CU assigns collaborative computation tasks among eligible vehicles and RSUs, schedules tasks and allocates computation resources to help the AV in obtaining real-time occlusion information. In other words, regardingless the road environments the vehicles drive through, the perception task offloading with collaborative computation is 'triggered' by the occlusions, that is, the nearby vehicles and RSUs, which are coordinated by the CU, implement collaborative computation for the vehicles once they have RoI perception requests. Hence, the practical road environments hardly exert influence on the decision-making process perception task offloading with collaborative computation. In this line, theoretical analysis such as in Section III-C and Section III-D, which are derived based on the generalized model in Fig. 3, can apply to other scenarios in various automotive environments.

C. Vehicle Movement

The AV's movement affects the selection of co-perceivers in the autonomous driving system. As the AV moving on the road, its occlusions change. For example, in Fig. 3, occlusion $\mathcal{R}_b$ is no longer in the perceptive range of RSU u_l, once AV v_0 moves into the coverage of RSU u_h. In this case, RSU u_l is not suitable for for being a co-perceiver anymore. In addition, the vehicle mobility problem exists in the collaborative computation. For example, if the co-perceiving AV accelerates away from the AV's communication range, the perceptive information will be lost because of the disconnected V2V link. Vehicular mobility is included in our work to settle these problems. Note that there is a CU caching the positional information of all AVs. Let $\mathcal{V}_f$ denote the AVs ahead of v_0 in $\mathcal{R}_{RoI}$, and AVs behind v_0 in $\mathcal{R}_{RoI}$ is represented as $\mathcal{V}_b$. Since $\mathcal{R}_{RoI}$ is either in or partly in an RSU's coverage range, we consider two neighboring RSUs, namely u_l and u_h. According

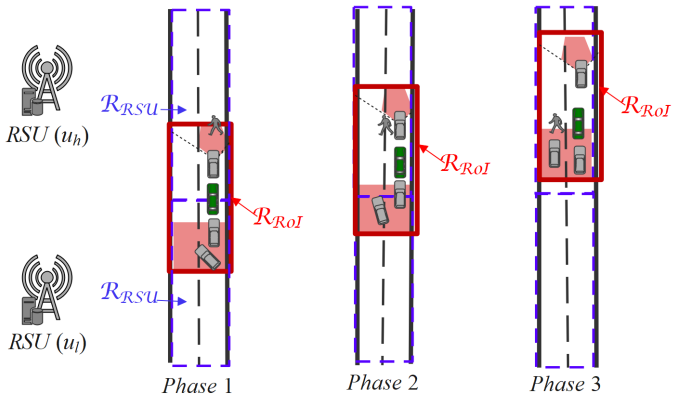

Fig. 4. Three-phase movement of AVs. In phase 1, $\mathcal{R}_b$ is about to move in u_h's perceivable range, but is still covered by u_l. In phase 2, part of $\mathcal{R}_b$ is perceived by the RSU u_h. In phase 3, all the areas in $\mathcal{R}_{RoI}$ is under u_h's perceivable range.

to the relative position of v_0 and RSUs, the movement of AV v_0 is described as a three-phase process, as shown in Fig. 4.

By assigning tasks among AVs and RSUs that are capable of perceiving $\mathcal{R}_f$ and $\mathcal{R}_b$, we define the task assignment function as $y = AS(x)$, indicating that collaborative task in $\mathcal{C}_{RoI}$ is allocated to y that can be a nearby AV or RSU. v_0 is capable of handling task $\mathcal{C}_0$ in $\mathcal{R}_0$, while the areas blinded by front AVs can be assigned to these AVs. The assignments of $\mathcal{C}_0$ and $\mathcal{C}_f$ are expressed as follows

$$v_0 = AS(\mathcal{C}_0), \tag{3}$$

$$\{v_i; \forall\, v_i \in \mathcal{V}_f \cap \mathcal{V}_{V2V}\} = AS(\mathcal{C}_f), \tag{4}$$

where $\mathcal{V}_{V2V}$ is the set of AVs within v_0's communication range.

Phase 1: $\mathcal{R}_b$ remains in u_l's coverage zone. Part of $\mathcal{R}_{RoI}$ is in the perceptive range of RSU u_h, while the dead area $\mathcal{R}_b$ behind AV v_0 stays in RSU u_l's sensing range. RSU u_l is eligible for task $\mathcal{C}_b$. Besides, any of AVs in $\mathcal{V}_b$ is qualified for collaborative task $\mathcal{C}_b$, as long as it is in the communication range of v_0.

$$\{v_i; \forall\, v_i \in \mathcal{V}_b \cap \mathcal{V}_{V2V}\} \,\|\, \{u_l\} = AS_{\rho=1}\left(\mathcal{C}_b\right), \tag{5}$$

where ρ is the phase indicator, $\rho = \{1, 2, 3\}$.

Phase 2: $\mathcal{R}_b$ is partly in u_l's coverage zone. In this phase, v_0 keeps moving and part of $\mathcal{R}_b$ is under the coverage of u_h, meanwhile the rest remains in u_l's sight of view. $\mathcal{C}_b$ can be handled by RSUs u_h and u_l collaboratively, or by the AV in $\mathcal{V}_b \cap \mathcal{V}_{V2V}$.

$$\{v_i; \forall\, v_i \in \mathcal{V}_b \cap \mathcal{V}_{V2V}\} \,\|\, \{u_l \cup u_h\} = AS_{\rho=2}\left(\mathcal{C}_b\right). \tag{6}$$

Phase 3: $\mathcal{R}_b$ is in u_h's coverage zone. v_0 keeps moving and $\mathcal{R}_{RoI}$ is completely in the sensing range of u_h. u_h can manage perception task $\mathcal{C}_b$ alone, or it can be handled by the AV in $\mathcal{V}_b \cap \mathcal{V}_{V2V}$.

$$\{v_i; \forall\, v_i \in \mathcal{V}_b \cap \mathcal{V}_{V2V}\} \,\|\, \{u_h\} = AS_{\rho=3}\left(\mathcal{C}_b\right). \tag{7}$$

According to the movement of v_0, we choose a set of co-perceivers among RSUs and AVs to assist v_0 in accomplishing perception tasks. Let Ω denote the set of the selected co-perceivers, which satisfies

$$\Omega = AS\left(\mathcal{C}_0\right) \cup AS\left(\mathcal{C}_f\right) \cup AS_{\rho=1,2,3}\left(\mathcal{C}_b\right). \tag{8}$$

Here $\Omega = \Omega_V \cup \Omega_U$, Ω_V and Ω_U are the set of AVs and RSUs assigned collaborative tasks, respectively.

D. Computation Offloading

To inform nearby AVs in Ω_V of collaboration, v_0 needs to transmit an instruction. As for RSUs in Ω_U, we assume that RSUs are connected with the CU through backbones, and thus have the collaborative task assignment decision copied in its cache. In this way, the perception task $\mathcal{C}$ is activated. The sensed data can be pre-processed by coordinate transformation engine, so that the perspective differences are eliminated [5]. The latency of instruction transmission and coordinate transformation is omitted, since the size of instruction is usually several bits and the transformation delay is negligible [7]. The perception task includes data collecting and computing. The perspective image is collected locally by visual sensors equipped on AVs and RSUs. We denote the collected data size as L. The data collection delay is not considered in our model since it is as low as a few milliseconds [28].

In order to provide comprehensive and accurate information for the motion planner, the collected data needs to go through localization engine, object detector and tracker for computing. To compute these tasks in real-time consumes a significant amount of computing resources, which may sometimes exceed the AV's computing power. By offloading tasks to RSUs that are equipped with edge servers, the computing delay is cut down, and the computing resources in the autonomous driving system are efficiently utilized.

1) Data Transmission: To offload perception tasks, the sensory data first needs to be transmitted from the AV's visual sensor to nearby AVs or the RSU. However, the amount of data is up to 2 GB/s, which incurs unaffordable transmission delay. To address this issue, we introduce the background subtraction technique, which separates the dynamic foreground objects from the static background of a scene [39]. The background is the same in each frame and is cached in RSUs beforehand. During the transmission process, only the extracted dynamic foreground needs to be transferred. Compared to the raw sensory image, the transmitted workload is notably reduced, thereby saving uploading time. We denote the ratio of the reduced transmission load to the original one as r^{sep}, and define M^{sep} as the CPU cycle requirement of the separation technology. Accordingly, the time spent on video separation is calculated as

$$D_{v_i}^{sep} = \frac{LM^{sep}}{f_{v_i}}, v_i \in \Omega_V. \tag{9}$$

Let $D_{v_i \to u_j}^{\text{trans}}$ denote the delay for transmitting data from AV v_i to RSU u_j. Then, we have

$$D_{v_i \to u_j}^{\text{trans}} = \frac{Lr^{\text{sep}}}{R_{v_i \to u_j}}, v_i \in \Omega_V, u_j \in \mathcal{U}, \tag{10}$$

where Lr^{sep} is the reduced data size, $\mathcal{U} = \{u_l, u_h\}$ is the set of RSUs, and $R_{v_i \to u_j}$ is the transmission rate between AV v_i

and RSU u_j. As such, we consider a multi-vehicle wireless interference model in uplink transmission. The transmission rate $R_{v_i \to u_j}$ is calculated as

$$R_{v_i \to u_j} = B \log_2 \left(1 + \frac{p h_{ij}}{\sigma_j^2 + \sum\limits_{v_k \in \mathcal{V} \setminus v_i} p h_{kj}} \right), \quad v_i \in \Omega_V, u_j \in \mathcal{U}, \tag{11}$$

where $v_k \in \mathcal{V} \setminus v_i$ is the set of AVs accessed to RSU u_j the same as AV v_i. p, σ_j^2 are the transmitting power of AV and the noise power, respectively. The term $h_{ij} = d_{ij}^{-\alpha_1}$ is the channel gain determined by the path-loss exponent $-\alpha_1$ and the distance d_{ij} between AV v_i and RSU u_j. d_{ij} is calculated as

$$d_{ij} = \sqrt{{d_{ij}^{'}}^2 + H^2}, \tag{12}$$

where $d_{ij}^{'}$ is the distance from the current position of AV v_i to the location of RSU u_j, and H is the height of RSU.

2) Task Computing: The offloading decision for collaborative AVs in Ω_V is described as

$$\mathcal{X} = \left\{ \mathcal{X}_1, \ldots, \mathcal{X}_i, \ldots, \mathcal{X}_{|\Omega_V|} \right\}, \tag{13}$$

where $\mathcal{X}_i$ is the offloading decision of AV v_i, $\mathcal{X}_i = \left\{ x_i^{\text{loc}}, x_i^{\text{det}}, x_i^{\text{track}} \right\}$. The elements in X_i are binary variables and represent the AV v_i's offloading decision for localization, object detection and tracking, respectively. $\mathcal{X}_i = 1$ if any of the variables in $\mathcal{X}_i$ is 1; $\mathcal{X}_i = 0$ indicates the perception tasks are computed locally at AV v_i. The completion delay of AV v_i's perception tasks is calculated as

$$\begin{aligned} D_{v_i} = &\max \left\{ D_{v_i}^{\text{loc}}, D_{v_i}^{det} + D_{v_i}^{\text{track}} \right\} \\ &+ \mathcal{X}_i D_{v_i \to u_j}^{\text{trans}}, v_i \in \Omega_V, u_j \in \mathcal{U}, \end{aligned} \tag{14}$$

where $D_{v_i}^{\text{loc}}$, $D_{v_i}^{\text{det}}$ and $D_{v_i}^{\text{track}}$ are the computing delay for localization, object detecting and tracking, respectively. Task dependency exists between object detecting and object tracking, since the output of the former one is the input of the latter. Localization task is computed in parallel with object detecting and tracking. Specifically, the computing delay of these perceptive tasks is calculated as

$$D_{v_i}^* = \frac{x_i^* L M^*}{\mu_{v_i}^* f_{u_j}} + \frac{(1 - x_i^*) L M^*}{f_{v_i}}, v_i \in \Omega_V, u_j \in \mathcal{U}, \tag{15}$$

where $D_{v_i}^*$ is the computation delay indicator of v_i's perception task, $D_{v_i}^* \in \left\{ D_{v_i}^{loc}, D_{v_i}^{det}, D_{v_i}^{track} \right\}$, and x_i^* is the task offloading indicator, $x_i^* \in \left\{ x_i^{\text{loc}}, x_i^{\text{det}}, x_i^{\text{track}} \right\}$. M^* is computation intensity indicator, $M^* \in \left\{ M^{loc}, M^{det}, M^{track} \right\}$, where M^{loc}, M^{det} and M^{track} are the computation intensity of localization, detection and tracking, respectively. $\mu_{v_i}^*$ is the resource allocation indicator of vehicle v_i, $\mu_{v_i}^* \in \left\{ \mu_{v_i}^{loc}, \mu_{v_i}^{det}, \mu_{v_i}^{track} \right\}$, where $\mu_{v_i}^{loc}$, $\mu_{v_i}^{det}$ and $\mu_{v_i}^{track}$ are the fractions of RSU u_j's computing power allocated to AV v_i for executing localization, detection and tracking tasks, respectively.

We define the total fraction of computing resource assigned to AV v_i as μ_{v_i}, which can be calculated as

$$\mu_{v_i} = \mu_{v_i}^{loc} + \mu_{v_i}^{det} + \mu_{v_i}^{track}, v_i \in \Omega_V. \tag{16}$$

Meanwhile, for each computing task of AVs in Ω_V, the latency should not exceed its delay requirement τ

$$\frac{L M^*}{x_i^* \mu_{v_i}^* f_{u_j} + (1 - x_i^*) f_{v_i}} \leq \tau, v_i \in \Omega_V, u_j \in \mathcal{U}. \tag{17}$$

If RSU u_j is in Ω_U, i.e., it is assigned a perception task, a portion of u_j's computing resources should be saved for the execution of its own tasks. We denote the overall faction of computing resources saved for RSU u_j as

$$\mu_{u_j} = \mu_{u_j}^{loc} + \mu_{u_j}^{det} + \mu_{u_j}^{track}, u_j \in \Omega_U. \tag{18}$$

Similar to (14), the completion delay of RSU u_j is expressed as

$$D_{u_j} = \max \left\{ D_{u_j}^{loc}, D_{u_j}^{det} + D_{u_j}^{track} \right\}, u_j \in \Omega_U. \tag{19}$$

Accordingly, the computing delay of RSU u_j's perception tasks is calculated as

$$D_{u_j}^* = \frac{L M^*}{\mu_{u_j}^* f_{u_j}}, u_j \in \Omega_U, \tag{20}$$

where $D_{u_j}^*$ is the computation delay indicator of u_j's perception task, $D_{u_j}^* \in \left\{ D_{u_j}^{loc}, D_{u_j}^{det}, D_{u_j}^{track} \right\}$, and $\mu_{u_j}^*$ is the resource allocation indicator of RSU u_j, $\mu_{u_j}^* \in \left\{ \mu_{u_j}^{loc}, \mu_{u_j}^{det}, \mu_{u_j}^{track} \right\}$.

Similar to (17), the computing delay of RSU in Ω_U follows the constraint

$$\frac{L M^*}{\mu_{u_j}^* f_{u_j}} \leq \tau, u_j \in \Omega_U, \tag{21}$$

where $\mu_{u_j}^*$ is the resource allocation indicator of RSU u_j, $\mu_{u_j}^* \in \left\{ \mu_{u_j}^{loc}, \mu_{u_j}^{det}, \mu_{u_j}^{track} \right\}$. The allocated resource fractions satisfies

$$\mathrm{I}\left(u_j \in \Omega_U\right) \mu_j + \sum_{i=1}^{|\Omega_{Vj}|} \mu_i = 1, u_j \in \mathcal{U}, \Omega_{Vj} \in \Omega_V, \tag{22}$$

where $\mathrm{I}(\cdot)$ is an indicator function, which equals 1 if the event is true and 0 the other way. Ω_{Vj} is the set of collaborative AVs under the coverage of u_j. (22) ensures that the fractions of the allocated computing resources add up to 1.

IV. Problem Formulation

In this section, we formulate the joint collaborative task assignment, offloading and resource allocation optimization as a MINLP problem, the goal is to minimize the completion delay of the collaborative computation task. The problem of joint collaborative task assignment, offloading and resource allocation is three-fold. The first one is how to assign collaborative tasks so that v_0 can achieve a full $\mathcal{R}_{RoI}$ awareness. The other two are which tasks to offload and how many resources to allocate, such that the completion delay is minimized. On top of that, the optimization problem is formulated as

$$\begin{aligned} \mathbf{P0}: &\min_{\Omega, \mathcal{X}, \mu} \max \left\{ D_{v_i}, D_{u_j} \right\}, \forall\, v_i \in \Omega_V, u_j \in \Omega_U \\ &s.t. \quad (8), (17), (21), \text{ and } (22). \end{aligned} \tag{23}$$

The spatial constraint (8) guarantees that all areas in $\mathcal{C}_{RoI}$ are perceived. Constraints (17) and (21) indicate the delay

requirements. (22) denotes that the sum of all the allocated portions of an RSU's computing resources is equivalent to 1. The properties of problem $\mathbf{P0}$ are expressed in the following lemmas.

Lemma 1: $\mathbf{P0}$ is a mixed-integer nonlinear programming problem.

Proof 1: The task offloading decision $\mathcal{X}$ is binary-valued and the variables in the resource allocation strategy μ are continuous, $\mathbf{P0}$ is a mixed-integer problem. The objective function $\max\left\{D_{v_i}, D_{u_j}\right\}$ is a nonlinear function of the decision variables. Hence, the formulated problem is a mixed-integer nonlinear problem. This completes the proof.

Lemma 2: Problem $\mathbf{P0}$ is non-convex.

Proof 2: Please refer to Appendix A.

In addition, Problem $\mathbf{P0}$ is a NP-hard combinatorial problem, as it includes MILP [40] and its solution usually requires searching enormous search trees or even worse, is undecidable [41]. The existing methodologies for MINLP include deterministic algorithms and heuristic methods. Typical deterministic algorithms include branch and bound (B&B), outer approximation (OA), and hybrid techniques, which show their advantage in solving convex programming problem. Nonetheless, deterministic algorithm tends to lose its advantage for in MINLP problem, as it requires solving the continuous relaxation of $\mathbf{P0}$ iteratively, which consequently increases the number of non-convex terms and the non-convexity of problem $\mathbf{P0}$. Heuristic methodologies include nature-inspired algorithms, genetic algorithms, etc. Heuristics are able to obtain good feasible solution in real time when the search tree has grown too large. However, global optima cannot be typically guaranteed.

To overcome the above-mentioned shortcomings, we design an efficient two-layer (TL) algorithm which integrates the merits of both heuristic algorithm and deterministic algorithm. To specify, in the outer layer, a binary intelligent firefly algorithm (BIFA) is devised for the optimal integer solutions, i.e., Ω and $\mathcal{X}$. Given the known information of integer variables, MINLP is reduced to a serial of NLP problems, an NLP solver works in the inner layer to calculate the continuous solution, i.e., μ. Compared to deterministic algorithm, the proposed TL algorithm settles the integer solutions without relying on variable relaxation, which would otherwise increase the non-convexity of $\mathbf{P0}$. Different from heuristic algorithms, the TL algorithm solves the continuous variables efficiently with the known gradient information.

V. Solutions

In this section, we present the design of the two-layer binary intelligent firefly algorithm (TL-BIFA), with the goal of solving the MINLP of joint collaborative task assignment, offloading and resource allocation. The proposed TL-BIFA works as follows. In the outer layer, an enhanced version of firefly algorithm is devised to find the best co-perceiver set and the feasible offloading solutions. In particular, concerning the nature of easily being trapped in local optima, the idea of the *survival of the fittest* law is applied to the basic firefly algorithm, ensuring that a firefly will only evolve towards fireflies with high brightness. With the integer variables fixed in outer layer, the MINLP problem is reduced to a serial of NLP problems with the known gradient information, the non-convexity is thereby reduced. The NLP problems are resolved by applying the Karush-Kuhn-Tucker (KKT) conditions.

The reason we use FA to settle the integer variables in the MINLP problem is threefold. First, the stringent delay requirement imposed by collaborative computation task in automotive environments entails an outer layer heuristic algorithm that can approach the global optima efficiently. The task assignment and offloading strategies are critical to the system's reliability and real-time performance. FA, as a very powerful technique based on global communications among the fireflies, can solve constrained NP-hard optimization problems with high efficiency. Second, FA has a simple operation and can be easily implemented. In FA, the generations of new solutions are updated by random walk and attraction of the fireflies. Different from Ant Colony Optimization (ACO) and many more bio-heuristic algorithms, FA improves solutions without relying on complex rules. Third, the BIFA improves its efficiency in approaching the global optima by applying the survival of the fittest law to generation updating, which makes it overcome the bio-heuristic algorithms' common nature of being trapped in local optima.

A. BIFA For Collaborative Task Offloading

The task offloading decision is a $(|\Omega_V|+2)\times 3$ matrix, where each row indicates a co-perceiver. Three columns respectively represent a perception task, namely, the object localization, object detection and object tracking. As such, task offloading decision $\mathcal{X}$ is written as

$$\mathcal{X} = \begin{bmatrix} x_1^{loc} & x_1^{det} & x_1^{track} \\ \vdots & \vdots & \vdots \\ x_{|\Omega_V|}^{loc} & x_{|\Omega_V|}^{det} & x_{|\Omega_V|}^{track} \\ \mathrm{I}(u_l \in \Omega_U) & \mathrm{I}(u_l \in \Omega_U) & \mathrm{I}(u_l \in \Omega_U) \\ \mathrm{I}(u_h \in \Omega_U) & \mathrm{I}(u_h \in \Omega_U) & \mathrm{I}(u_h \in \Omega_U) \end{bmatrix}, \quad (24)$$

where the first $|\Omega_V|$ rows are the offloading decisions of collaborative AVs, each x_i is binary-valued. The last two rows indicate the RSUs' collaborative tasks. For example, $\mathrm{I}(u_h \in \Omega_U) = 1$, if RSU u_h is assigned with collaborative tasks. In this case, u_h has to save part of computing resources for the three tasks. Otherwise, $\mathrm{I}(u_h \in \Omega_U) = 0$. In short, the co-perceiver set Ω determines the number of rows in $\mathcal{X}$ and the value in the last two rows, which follows the spatial constraint (8). Note that there are possibly multiple eligible co-perceiver sets. We settle the set with the most computing resources as Ω in the outer layer of TL-BIFA, as the co-perceivers are able to provide faster computation results for v_0.

We define a population of fireflies as

$$\mathbb{X} = \{\mathcal{X}_1, \cdots, \mathcal{X}_m, \cdots, \mathcal{X}_M\}, \quad (25)$$

where M is the number of fireflies in the population, $\mathcal{X}_m$ indicates a collaborative offloading strategy. The light intensity of a firefly depends on the objective function, i.e., task completion delay. However, the brightness seen by another

firefly is affected by the distance between the two. A firefly tends to be attracted to an adjacent firefly with a strong light. We define the light intensity at the source as

$$\mathsf{L}_m = \frac{1}{\max\left\{D_{v_i}, D_{u_j}\right\}}, v_i \in \Omega_V, u_j \in \Omega_U. \tag{26}$$

The light intensity L_{ab} of firefly a observed from the perspective of firefly b varies according to the inverse square law [42]

$$\mathsf{I}\left(d_{ab}\right) = \mathsf{I}_a \exp\left(-\gamma d_{ab}^2\right), \tag{27}$$

where d_{ab} is the distance between fireflies a and b, which is calculated as the euclidean distance between $\mathcal{X}_a$ and $\mathcal{X}_b$. γ is the degree of absorption since the light is also absorbed by air. Besides, a firefly's attractiveness is determined by the light intensity seen by neighboring fireflies [42], which is expressed as

$$\beta\left(d_{ab}\right) = \beta_0 \exp\left(-\gamma d_{ab}^2\right), \tag{28}$$

where β_0 is the attractiveness when $d_{ab} = 0$.

According to the above metrics, the position of a firefly (i.e., the solution of an offloading strategy) is updated according to

$$\mathcal{X}_m = \mathcal{X}_m + \beta\left(\mathcal{X}_q - \mathcal{X}_m\right) + \alpha_2 \epsilon. \tag{29}$$

The next position $\mathcal{X}_m$ of firefly m is related to its last position (i.e., the first term), the attraction (i.e., the second term) and a randomization term $\alpha_2\epsilon$. α_2 is a randomization parameter and ϵ is a random number drawn from a uniform distribution [0, 1]. The third term in (29) is introduced to avoid being stuck in local optima. $\mathcal{X}_m$ is discretized after position updating, since a feasible offloading solution is binary-valued.

The process of the proposed BIFA algorithm for collaborative task offloading is presented in Algorithm 1. Each offloading decision in $\mathbb{X}$ is updated according to (29), correspondingly the population is changed. Even so, whether the population is upgrading or retrograding is hard to guarantee, since there are situations where a firefly is attracted to another one, and is taken away from the global optimum, merely because it is graphically close. We purposely make Modifications to firefly algorithm to avoid such cases, hence an enhanced version of firefly algorithm is designed. In BIFA, we rank population $\mathbb{X}$ in descending order according to L_m before position updating. In each generation, a firefly moves only towards top ϕM fireflies, where ϕ is percentage of the number of elite fireflies to the population. In so doing, it is ensured that the population is upgraded after each iteration.

B. NLP Solver For Computing Resource Allocation

With collaborative task offloading strategy $\mathcal{X}$, together with Ω, is fixed in outer layer, integer variables are settled. **P0** thereby contains only a set of continuous variables, i.e., $\mu \in [0, 1]$. The MINLP **P0** is reduced to the NLP problem, which is denoted by

$$\begin{aligned} \mathbf{P1}: &\min_{\mu} \max\left\{D_{v_i}, D_{u_j}\right\}, \forall\, v_i \in \Omega_V, u_j \in \Omega_U. \\ &s.t. \quad (17), (21), (22). \end{aligned} \tag{30}$$

There are $k_1 = \sum\sum_{i=1}^{|\Omega_V|}\mathcal{X}$ inequality constraints representing the delay requirements of AV's offloaded tasks, and $k_2 = \sum\sum_{|\Omega_V|+1}^{|\Omega_V|+2}\mathcal{X}$ inequality constraints indicating the delay requirement of RSU's computing tasks. The total number of inequality constraints $k_3 = \sum\sum\mathcal{X}$. Additionally, two equality constraints ensures that the sum of each RSU's allocated resource fractions is 1. For clarity, we define the inequality constraints as

$$g_k = \frac{LM}{x_i \mu_{v_i} f_{u_j} + \left(1 - x_i\right) f_{v_i}} - \tau\,, k = 1, \cdots, k_1. \tag{31}$$

$$g_k = \frac{LM}{\mu_{u_j} f_{u_j}} - \tau,\ k = 1, \cdots, k_2. \tag{32}$$

The equality constraints are expressed as

$$h_k = \mathrm{I}\left(u_j \in \Omega_U\right)\mu_j + \sum_{i=1}^{|\Omega_{Vj}|}\mu_i - 1\,, k = 1, 2. \tag{33}$$

Then we obtain the optimization problem:

$$\min_{\mu} f(\mu) = \max\left\{D_{v_i}, D_{u_j}\right\}, \tag{34}$$

subject to

$$h_k = 0,\ k = 1, 2 \quad \text{and} \quad g_k \leq 0\, k = 1, \ldots, k_3.$$

P1 is a non-linear optimization problem with equality and inequality constraints. To solve **P1**, we define the Lagrangian operator as

$$\mathcal{L}(\mu, e, q) = f(\mu) + \sum_{k=1}^{2} e_k h_k + \sum_{k=1}^{k_3} q_k g_k. \tag{35}$$

We can find a local optimum μ^* if there exists a unique $\mathbf{q}^*$ satisfying

$$\bigtriangledown_{\mu}\, \mathcal{L}(\mu^*, e^*, q^*) = 0, \tag{36}$$

$$q_k^* \geq 0, \text{ for } k = 1, \ldots k_3, \tag{37}$$

$$q_k^* g_k(\mu^*) = 0, \text{ for } k = 1, \ldots k_3, \tag{38}$$

$$g_k(\mu^*) \leq 0 \text{ for } k = 1, \ldots, k_3, \tag{39}$$

$$h_k(\mu^*) = 0 \text{ for } k = 1, 2, \tag{40}$$

$$\mathbf{y}^t \bigtriangledown_{\mu\mu}\, \mathcal{L}(\mu^*)\mathbf{y} \geq 0,\ \forall \mathbf{y} \text{ orthogonal to } \bigtriangledown_{\mu}\, g(\mu^*). \tag{41}$$

Constraints (36)-(41) are the KKT conditions associated with problem **P1**. (36) and (37) ensure that μ^* is an extreme point. (41) makes sure that $\bigtriangledown_{\mu\mu}\mathcal{L}(\mu^*)$ is a positive semi-definite matrix, and μ^* is a local optimum. Constraints (39) and (40) are the original equality and inequality constrains, respectively. (38) is the complementary slackness, which indicates the impact of inequality constraints on the optima. The process of NLP Solver is presented in Algorithm 2.

Lemma 3: Eq. (38) is the prerequisite of an optimum μ^*.

Proof 3: If we find a local optimum μ^* that lies within the feasible region, i.e., $g(\mu^*) \leq 0$, a constrained local minimum is the same for an unconstrained local minimum, and thus $q = 0$. If μ^* is out of the feasible region, the inequality constraints are effective. Then, we have $g(\mu^*) \leq 0$. The optimal solution occurs when $\bigtriangledown_{\mu} f(\mu)$ and $\bigtriangledown_{\mu} g(\mu)$ are parallel

$$-\bigtriangledown_{\mu} f(\mu) = \lambda \bigtriangledown_{\mu}\, g(\mu). \tag{42}$$

In other words, if resource allocation μ^*, together with the dual parameter pair e^* and q^*, satisfies the KKT conditions (36)-(41), μ^* is the optimal resource allocation. This completes the proof.

Algorithm 1: Outer Layer: BIFA Algorithm

Input: the AV's current phase ρ, FoV F_i and $\mathcal{R}_{RoI}$, AVs $\mathcal{V}$, RSUs $\mathcal{U}$, RSU's coverage range $\mathcal{R}_{RSU}$ and computing resource f_{u_j}, the set of AVs whithin v_0's communication range, $\mathcal{V}_{V2V}$, AV's computing power f_{v_i} and task characteristics.
Output: collaborative task assignment Ω, task offloading decision $\mathcal{X}$ and RSUs' resource allocation μ.

OUTER LAYER FOR Ω, $\mathcal{X}$
Set the co-perceiver set that satisfies (8) and has the most computing resources as Ω;
Randomly generate the first population of fireflies $\mathbb{X}$ according to Ω ;
while $z \leq$ *maximum number of generations* **do**
  Rank fireflies in descending order according to lightness;
  Record the best firefly $\mathcal{X}_{best}$ and the corresponding Ω_{best} in this population;
  for $m = 1$ *to* M **do**
    for $a = 1$ *to* ϕM **do**
      if $L_a > L_m$ **then**
        Calculate the euclidean distance d_{am} between $\mathcal{X}_m$ and $\mathcal{X}_a$;
        Calculate the attractiveness according to (28);
        Update firefly $\mathcal{X}_m$ in the z-th generation according to (29);
        Normalize firefly $\mathcal{X}_m$ to interval $[0, 1]$;
        Discretize firefly $\mathcal{X}_m$;
    INNER LAYER FOR μ
    Put k_3 inequality constraints and equality constraints (17), (21) and (22), and the objective function $f(\mu)$ into NLP solver;
    Record the optimal resource allocation μ for each firefly;
    Update the lightness of firefly m according to (26);
  Update the z-th generation of fireflies $\mathbb{X}$;
  Record the best firefly of the z-th generation as $\mathcal{X}_{best}$;
Return Ω, $\mathcal{X}$, μ.

C. Algorithm Complexity Analysis

In the two-layer binary intelligent firefly algorithm (TL-BIFA), the computational complexity of the initialization phase is $O(N)$, the position update of a firefly generation in the outer layer is $O(\phi M)$. With the integer variables settled in the outer layer, problem **P0** turns into the resource allocation problem **P1**, which is solved by the NLP solver in the inner layer with computational complexity of $O(k_3)$. Hence, the overall computational complexity of TL-BIFA is $O(N + \phi Z M_2 + Z M k_3)$, where Z is the maximum number of firefly generations.

Algorithm 2: Inner Layer: NLP Solver

Input: collaborative task assignment Ω, offloading decision $\mathcal{X}$.
Output: RSUs' resource allocation μ.

Generate matrices $\mathcal{X}_{ml}$ and $\mathcal{X}_{mh}$ to store the perception tasks assigned to RSU u_l and RSU u_h, respectively;
Initialize $\mathcal{X}_{ml}$ and $\mathcal{X}_{mh}$ according to Ω, $\mathcal{X}$;
calculate the total number of perception tasks that RSU u_l and RSU u_h needs to compute, record the numbers as k_l, k_h, respectively;
for *each RSU in* $\mathcal{U}$ **do**
  Define the inequality constraints according to (31) and (32);
  Define the equality constraint according to (33);
  Define the Lagrangian operator according to (35);
  Find the resource allocation μ according to the KKT conditions (36)-(41);
Return Ω, $\mathcal{X}$, μ.

VI. Evaluation and Discussion

A. Trace-based Simulations

We consider an unidirectional and two-lane road with a lane width of 3.75 *m*, the distance from the RSU to the centerline of the road is 15 *m* [7]. The sensing radius, as well as V2I transmission range, of an RSU is 200 *m* [43], and hence RSUs are deployed at a interval of 200 *m* to provide a full coverage. The height of RSU is 20 *m* [7]. There are $N = 10$ AVs scattered under the coverage of two neighboring RSUs, each AV is armed with a visual sensor. The camera equipped on AVs is Leopard Imaging LI-USB30-P031XB, and its FoV is $\mathrm{D}112^\circ \times \mathrm{H}89.6^\circ \times \mathrm{V}67.2^\circ$ [5]. The sensory data size of single visual frame is 100 *KB* with a resolution of 1280×720 pixels [29]. The computation intensity of perception tasks follows a uniform distribution in $[1,000, 2,640]$ *cycles/bit* [44], [45], and the delay requirement τ is 0.1 *s* [46]. Using dynamic foreground and static background separation technique, the ratio of reduced workload to the original data size r_{sep} is 0.85 [39].

According to Horizon Robotics' summary, a higher level of automated driving requires more orders of magnitude tera operations per second (*TOPS*), namely, 10 *TOPS* for L2 autonomy, 30 to 60 *TOPS* for L3, more than 100 *TOPS* for L4, and around $1,000$ *TOPS* for L5. Our autonomous driving system is based on Huawei MDC 810 computing platform and could achieve 400 *TOPS* of computing power. Due to the heterogeneity of AV users' computing demand, the computing resource of AVs is drawn from a random distribution in $[3, 5]$ *GHz* [9], and the computing power of each RSU is 20 *GHz* [47]. Collaborative environment perception are carried out in RoI to ensure autonomous driving safety, the length of $\mathcal{R}$ is 100 *m* [48] and V2V communication range is 70 *m* [49]. In the perception task offloading system, the bandwidth assigned to each collaborative AV is $B = 1$ *GHz*, the noise power σ_{j}^2 is 10^{-13} *W* and the path-loss exponent α_1 is 4 [47].

In BIFA, there are 50 generations of firefly population and the total number of fireflies in each population is 10. The initial attractiveness β_0 is set as 1. The randomization parameter $\alpha_2 = 1$.The parameter γ characterizes the variation of the attractiveness, determining the speed of the convergence and how the FA algorithm behaves. In our settings, γ is set to be varied from 0.1 to 10.

B. Metrics and Baselines

We conduct the evaluation based on three *metrics*, i.e., delay, resource utilization and the completion of RoI perception.

1) Delay: Delay measures the efficiency of collaborative task, and is related with the safety level of automated driving systems [24].

To evaluate the efficiency of TL-BIFA, we compare the algorithm with two-layer random search (TL-RS) and two-layer firefly algorithm (TL-FA). The reason we choose TL-RS for comparison is that, an AV's RoI is limited, the number of the selected co-perceivers are small (e.g., several collaborative AVs and RSUs), TL-RS can find a relatively good solutions in given time. In TL-RS,a guarantee of optimality is sacrificed for finding a good offloading solution quickly. Compared to TL-BIFA, TL-FA updates fireflies according to distance, without considering objective function. Different from TL-FA, TL-BIFA upgrade the population by moving fireflies towards the best ϕM fireflies in the population. We investigate the impact of parameter ϕ on the convergence of TL-BIFA and the benchmark algorithms, by tuning up ϕ from 0.5 to 0.75. The results are shown in Fig. 5.

To investigate the performance of the outer-layer and inner-layer algorithms, we compare the proposed FA algorithm with two baselines, i.e., the ant colony optimization (ACO) algorithm and the whale optimization algorithm (WOA) [50], and we compare the NLP-solver with two baselines, i.e., the branch and bound (B&B) algorithm [51] and the Markov approximation (MA) algorithm [52]. Following the work [26], the exhaustive search (ES) algorithm is used to find the optimal bound. The results are shown in Fig. 6.

We evaluate the resource allocation performance by comparing the proposed scheme with four baselines, i.e., the online dynamic adaptive (ODA) scheduling algorithm [8], the federated Q-learning (FQL) algorithm [53], the Attention-weighted Recurrent Multi-Agent Actor-Critic (ARMAAC) algorithm [54] and the low complexity greedy based efficient searching (GES) algorithm [26]. The ODA algorithm optimizes task offloading decisions without considering resource allocation. Computing resources of vehicles and RSUs are allocated to shorten the response time under the FQL and ARMAAC schemes, respectively. GES makes resource allocation strategies for both vehicles and RSUs in order to meet the stringent delay requirement. The results are shown in Fig. 7.

2) Resource Utilization: Resource utilization refers to the proportion of consumed computing resources of the system. The higher the resource utilization is, the more computing resources are used for processing. We compare our perception offloading method with two baselines, i.e., local computing (LOCO) and edge offloading (EDO), in terms of delay and resource utilization. Under the LOCO scheme, the tasks are processed by AVs locally without computation offloading. In EDO, sensory data is outsourced to RSUs for processing. The results are shown in Fig. 8.

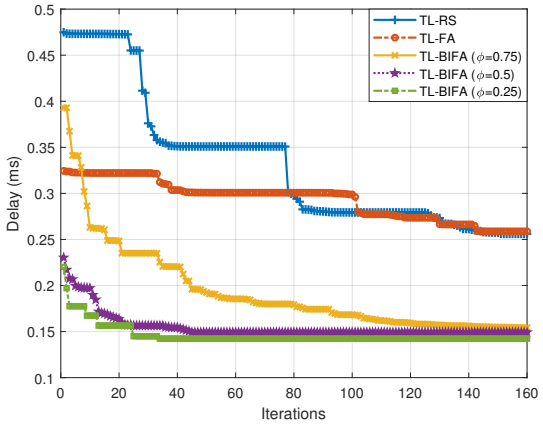

Fig. 5. Delay of algorithms with iterations.

We assess our data uploading method (see in Section III-D1) by comparing our dynamic foreground uploading mechanism with background subtraction (w/ BG Sub) to the raw sensory data uploading mechanism (w/o BG Sub). The results are shown in Fig. 9.

3) Offloading Ratio: Offloading ratio refers to the proportion of outsourced data to the overall computation workload. We investigate the impact of computation intensity on offloading ration under various computing power of AV. The results are shown in Fig. 10.

4) Completion of RoI Perception: We measure the performance of the collaborative computation scheme in terms of RoI completion ratio, which indicates the fraction of total sensed area to the entire RoI. When the ratio rises up to 1, the whole RoI is being perceived and hence the occlusions in $\mathcal{R}_{RoI}$ are eliminated. The higher the completion ratio of RoI is, the more knowledge about more areas in RoI the vehicle can obtain. In view of completion ratio, we compare the collaborative computation with infrastructure computation (INCO) and vehicle computation (VECO). Under the EDCO mode, only RSUs are the vehicle's co-perceivers, while in AVCO, only nearby AVs are able to assist AV in perceiving the occlusions. The results are shown in Figs. 11 and 12.

C. Efficiency of the Proposed TL-BIFA

Fig. 5 presents the performance of algorithms, including both the convergence speed and the optimal solutions. It can be seen that the solutions found by RS are the worst of the considered algorithms in the beginning of the iterations, this is because TL-RS generate solutions arbitrarily in feasible space in the beginning. On the contrary, better solutions are rendered by TL-FA and TL-BIFA at the same stage. Compared to TL-FA, TL-RS converges faster at around 150 iterations. In fact, TL-RS can make relatively good decisions after about 90 iterations. This is owing to the fact that the feasible space in our collaborative task offloading approach is not large, while TL-FA usually requires quite a number of generations and thus

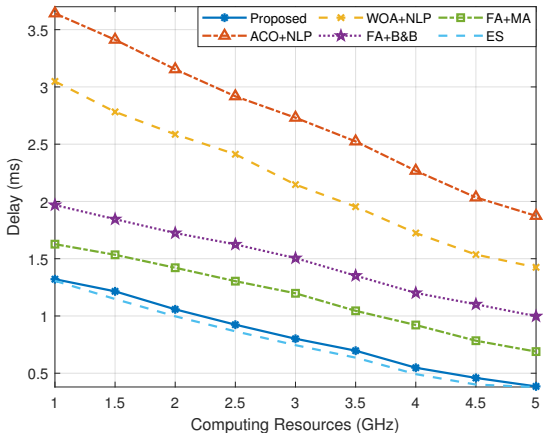

Fig. 6. Algorithm performance with various computing resources.

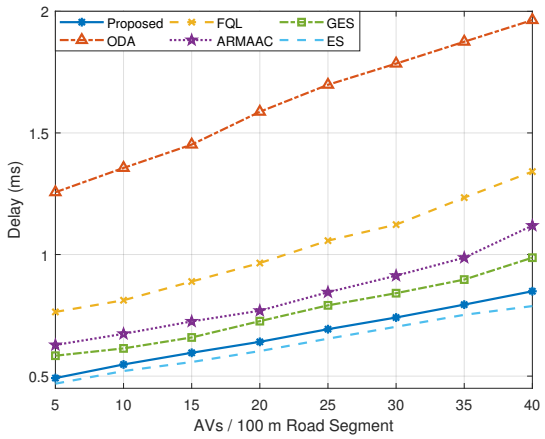

Fig. 7. Resource allocation scheme performance with vehicle density.

has a slow convergence speed. In essence, TL-FA is equivalent to TL-BIFA when ϕ is set to 1, in which case the survival-of-the-fittest mechanism is not activated. When ϕ is set to 0.75, the fireflies are updated towards the top 75 percent of the population, which prompts firefly population's evolution.

As ϕ is tuned down to 0.5, the proposed TL-BIFA converges quickly at iteration 35, and the optimal delay is 0.14 *ms*. The results of TL-BIFA with $\phi = 0.5$ and 0.25 are similar, ϕ only has a minor impact on TL-BIFA performance within the range of $[0.25, 0.5]$. However, significant improvement of convergence speed can be observed when ϕ jumps from 0.75 to 0.5. According to [20], the total tolerable delay for perception is 3 *ms* when the AV is driving at 50 km/h. This optimal delay is far less than the upper bound delay. The reason behind is that, with the decrease of ϕ, the screening condition becomes more stringent and the number of example solutions is cut down by 25%. Above all, the total completion delay of perception task is no larger than 0.5 *ms*. These results validate the efficiency and reliability of the perception task offloading scheme with collaborative computation.

Fig. 6 presents the performance of algorithms in the outer and inner layers. It can be seen that the delay gap between the proposed algorithm and ES is 0.056 *ms* with 5 *GHz* computing resources, the completion delay achieved by the proposed TL-BIFA algorithm is close to the optimal bound (e.g., the ES). The ACO+NLP method achieves the delay of 1.874 *ms*, which is greater than TL-BIFA by 386.75%. This is due to the fact that the evaporation rate in ACO directly impacts the global search ability, which can lead to reduction of global search ability and bring the algorithm to local convergence. On the contrary, the outer-layer FA has the ability to search for better perception task offloading by means of elite reproduction. The delay obtained by FA+B&B is 0.998 *ms*, which is greater than TL-BIFA by 159.22%. This is because, the B&B requires more branching computation for a resource allocation strategy, which makes it less computationally efficient in comparison with the NLP-solver. Given the delay requirements and offloading strategy, the NLP-solver can return the optimal resource allocation strategy in a much less time-consuming manner.

In addition, the outer-layer algorithm has a greater impact on the delay than the inner-layer algorithm. For example, when the computing resources are 3 *GHz*, the delay sees a 49.56% increase under the FA+MA scheme, which is less than that with WOA+NLP scheme by 118.44%. This is because, the outer-layer algorithm decides the upper bound of available resources for each task, while the inner-layer algorithm further improves the delay performance by allocating the computing resources among tasks based on offloading decision made in the outer layer.

D. Perception Task Offloading

Fig. 7 illustrates the delay versus the number of vehicles in every 100 m road segment among the proposed resource allocation scheme and the baselines. As seen from Fig. 7, the delay grows as the AV density increases, which verifies the fact that more surrounding vehicles cause more occlusions, and hence generate more collaborative computation tasks. The delay achieved by the proposed algorithm is close to the optimal bound (e.g., the ES), and is lower than that under the GES scheme by 16.25% when there are 40 vehicles in every 100 *m* road segment. This is because, the GES enables reverse offloading, bringing extra transmission delay for the dependent perception tasks. The ARMAAC reduces the delay by 19.84% compared to the FQL algorithm, since RSUs typically have more powerful computing capacity than vehicles, and can process more computation tasks. Reasonable allocation strategy of RSUs can speed up the collaborative computation process and shorten the completion delay. The ODA scheme incurs much higher delay compared to other algorithms, as it schedules tasks to reduce the delay without making optimal computing resource allocation strategy. Overall, the proposed algorithm makes better resource allocation strategies of both RSUs and vehicles such that the task completion delay is significantly reduced.

Fig. 8 depicts the impact of input workload (e.g., the sensory data) on delay and resource utilization under different offloading schemes. With the increase of sensory data, task completion delay grows (see the red curves). The LOCO obtains the largest delay, as it is prone to be easily affected by the input workload. The compromise might be uploading

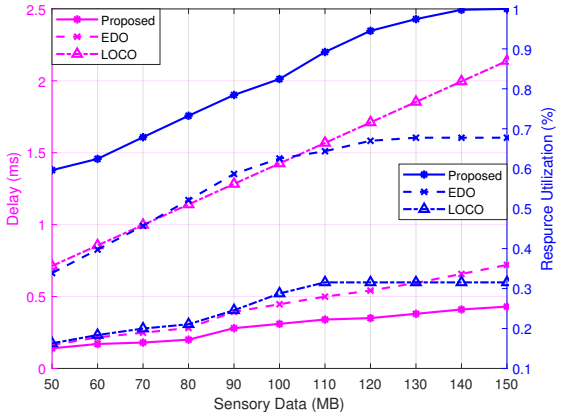

Fig. 8. Delay and resource utilization under different offloading schemes.

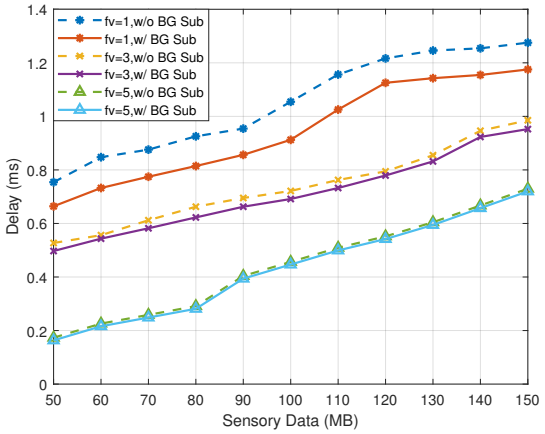

Fig. 9. Delay under different data uploading mechanisms.

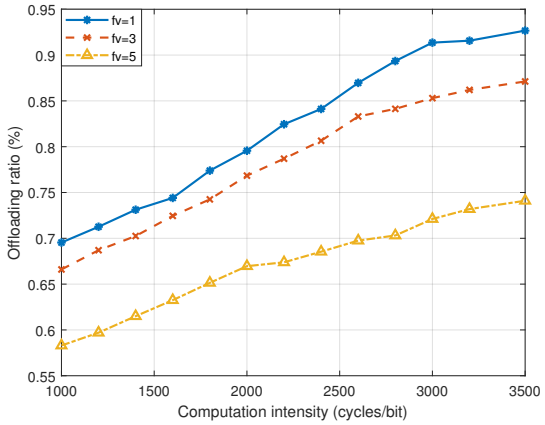

Fig. 10. Offloading ratio of power-varied AVs with computation workload.

all the data to the edge, i.e., EDO, since the edge server is equipped with considerable computing power. However, the transmission delay of EDO can be intolerable due to the huge amount of the perceived data. As shown in Fig. 8, as the data size increases, the delay gap between EDO and the proposed becomes wider. Note that the EDO and proposed offloading scheme see a sudden increase in terms of delay when the sensory data size rises from 80 to 90 *MB*. This is because, the performance of offloading schemes is studied based on the proposed TL-BIFA algorithm, the randomness that exists in the outer-layer algorithm does not promise the consistency or smoothness between solutions [55]. The proposed task offloading scheme shows its intelligence in flexibly choosing co-perceivers among RSUs and AVs, thereby reducing completion delay to the lowest level. This is also supported by the results of resource utilization (see the blue curves in Fig. 8). As the workloads rise, the resource utilization grows as well. This is because more computing resources are demanded by growing sensory data. It can be seen that the utmost utilization of LOCO is around 0.325, and that of EDO is 0.675, since the constrained resources of AVs and RSUs limit the maximal utilization under LOCO and EDO, respectively. The bottleneck of resource utilization is overcome by our proposed perception task offloading scheme, which can achieve a maximal utilization of 100%, meaning that all computing resources are utilized for processing when the computing demand rises to a certain level.

Fig. 9 illustrates the total latency of the collaborative computation with respect to input data size, concerning different uploading methods and AV's computing power. The uploading methods include dynamic foreground uploading with background subtraction (w/ BG Sub, solid line) and raw sensory data uploading (w/o BG Sub, dotted line). It can be seen that the completion delay of w/ BG Sub is always lower than that of w/o BG Sub. This is because the data size of the uploaded dynamic foreground is smaller compared to the original image after background subtraction technique. The delay gap under the two scheme grows with the decrease of AVs' computing power. When the input data is 150 *MB*, $f_v = 5$ *GHz*, the delay gap reaches up to 0.12 *ms*. This is because, as AVs' computing resources decline, more and more perception tasks have to seek computing support from RSUs, meaning that more data need to be transmitted. By exploiting background separation technique, transmission delay is significantly reduced. For example, when input data is 100 *MB* and $f_v = 5$ *GHz*, the delay is cut down by 22.2%. Besides, the delay is more affected by uploading method when f_v increases, which verifies the fact that fewer tasks need to be offloaded when AVs have ample computing power.

Fig. 10 depicts the impact of computation intensity on offloading ratio, which is the proportion of offloaded data size to overall data size. The higher offloading ratio is, the heavier workload is offloaded to the RSU. It can be observed that the offloading ratio rises with the increase of computation intensity, as RSUs often have more computing resources. Offloading ratio with $f_v = 1$ *GHz* stays high above 70%. This is because the computing power of AVs is too low to support computationally intensive perception tasks. When $f_v = 5$ *GHz*, AVs can deal with more computation workload locally without task offloading.

E. Collaborative Computation

Fig. 11 shows the completion of RoI under V2I computation (INCO) mode and collaborative (i.e., V2V+V2I) computation architecture. In INCO scheme, the AV can only collaborate

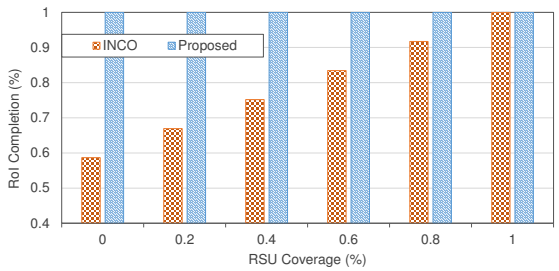

Fig. 11. RoI completion under V2I computation scheme.

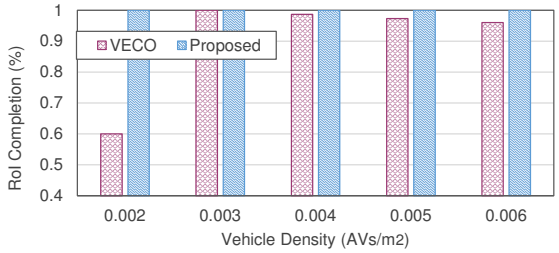

Fig. 12. RoI completion under V2V computation scheme.

with RSUs to perform perception task, while in collaborative computation, both nearby AVs and RSUs participate in perception. It can be seen that the RoI completion of collaborative computation mechanism is stabilized at 100%, implying that it is impervious to the RSU's coverage ratio in RoI. No matter how low the coverage ratio is, the proposed architecture is able to explore both V2V and V2I links. Even if RoI is visible to none of the RSUs, there are always nearby AVs capable of perceiving the environment. Contrarily, in INCO, the completion ratio is proportional to the RSU's sensing area in RoI. When the RSU covers the whole RoI, the completion ratio rises up to 1. Instead, if all of the RoI is not under the coverage range of RSUs, the completion ratio is as low as 0.586. This is because, there are 58.6% of RoI can still be perceived by v_0 itself without collaboration, while the rest areas (e.g., occlusions) remain unobserved.

The performance of V2V computation (VECO) system is shown in Fig. 12. With 0.002 AV density, the RoI completion ratio is as low as 0.6 under the VECO scheme. The reason is that, the number of available AVs in the vicinity is small, there are seldom AVs to collaborate with. With more AVs surrounding v_0, the number of accessible co-perceivers grows, then more AVs help to perceive RoI, and the completion rate increases to an utmost level. However, once the vehicle density exceeds a certain value (i.e., around 0.004 AVs/m^2 in our case), completion rate drops immediately. The reason is that, more occlusions on the rear region of the AV's RoI are caused by the increasing AVs, which are unperceivable to the front-camera equipped AVs. The performance of VECO relies on the sensing range of nearby AVs. When the AVs are densely-distributed, the unsupervised occlusions in VECO unveil the flaw of this architecture. Opposite to the VECO scheme, the completion performance stabilizes at 100%, which ensures the reliability of collaborative computation.

TABLE II
PERCEPTION TASK OFFLOADING PERFORMANCE UNDER VARIOUS COMPUTING PLATFORMS. PLT, CP AND CMPL ARE SHORT FOR COMPUTING PLATFORM, COMPUTING POWER AND ROI COMPLETION RATIO, RESPECTIVELY.

PLT	CP (TOPS)	FA	NLP-Solver	Delay (*ms*) ↓	CMPL (%) ↑
MQ4	2.5	×	×	16.39	18.82
		×	✓	13.21	26.48
		✓	×	12.26	29.67
		✓	✓	**10.28**	**35.73**
MQ5	24	×	×	9.64	42.59
		×	✓	7.13	49.73
		✓	×	6.45	58.94
		✓	✓	**5.62**	**69.78**
NVDX	30	×	×	6.74	60.32
		×	✓	5.25	74.97
		✓	×	3.98	82.57
		✓	✓	**2.37**	**88.32**
TSLHW	144	×	×	2.18	95.09
		×	✓	1.75	98.43
		✓	×	1.04	99.47
		✓	✓	**0.49**	**100.00**
HWNDC	400	×	×	0.35	100.00
		×	✓	0.27	100.00
		✓	×	0.19	100.00
		✓	✓	**0.14**	**100.00**

F. The Performance under Various Computing Platforms

Table II presents the completion delay and RoI completion ratio (CMPL) under different computing platforms (PLT). The level of autonomous driving is inseparable from the computing power (CP) of the chip [56]. Mobileye Q4 (MQ4) and Q5 (MQ5) provide 2.5 and 24 *TOPS* of computing power, respectively [57], for L1-L2 level automatic driving. NVIDIA Xavier processor delivers 30 *TOPS* for L3 autonomous driving. The computer of Tesla HW3.0 provides 144 *TOPS* computing power, aiming at autonomous levels 4 and 5. Huawei MDC 810 provides a maximum of 400 *TOPS*, which can support L4 autonomous driving. We evaluate the delay and RoI completion performance on the computing platforms with FA and NLP-solver working separately and jointly. While the FA and NLP-solver are not used, they are replaced by random offloading scheme and proportional resource allocation strategy, respectively. The perception delay requirement is set as 3 ms.

In Table II, as the computing power decreases, the delay rises, and the RoI completion ratio drops. For example, the delay is increased by 43.03%, and the RoI completion ratio is reduced by 17.73% when neither the FA nor the NLP-solver is adopted, and the computing power drops from 30 *TOPS* to 24 *TOPS*. This is because the computation tasks exceeding the tolerable perception delay are dropped, the vehicle hence does not receive the related occlusion information, and the RoI completion performance is degraded. When the computing power rises to 144 *TOPS*, the delay is lower than 0.5 *ms*, and the completion ratio remains at 100% when both of the algorithms are adopted. This indicates that the computing power of vehicle's onboard computing platform has a significant impact of task completion delay. Many of the vehicles' perception tasks can be handled locally and efficiently without seeking task offloading when the computing power grows. When the computing power increases to 400 *TOPS*, the delay is reduced to 0.14 *ms* under the proposed scheme. Compared to the resource allocation scheme, the task offloading algorithm has a larger impact on system performance. For example, FA

brings a 45.71% delay reduction when used alone, which is greater than that brought by NLP-solver by 22.85%. The reason is that, the outer-layer algorithm schedules where the tasks are executed, and hence determines the maximum value of available resources each task can obtain, while the inner-algorithm further improves the delay performance on the basis of offloading strategy made by the outer-layer algorithm.

VII. Conclusion

In this paper, we have developed a new framework of perception task offloading framework with collaborative computation for autonomous driving. By fully considering the vehicle mobility and task dependency, we formulate a joint problem of collaborative task assignment, offloading and resource allocation for minimizing task completion delay. The joint problem is formulated as MINLP, and a two-layer optimization method is presented. In the outer layer, BIFA is used to choose co-perceivers among RSUs and AVs and to offload collaborative tasks for AVs. In the inner layer, the NLP solver is applied to find the best resource allocation decision for RSUs. Experimental results demonstrate the superiority of our proposed method in raising the autonomous driving system's efficiency, safety and reliability.

In the future, we will devote our efforts in adapting collaborative task offloading system to more complex traffic roads like intersections and ramps. Besides, we will evaluate its performance by building prototypes in real-world automotive environments.

Appendix

A. Proof of Lemma 2

According to [41], if either the objective function or any inequality constraint in $\mathbf{P0}$ is non-convex, then we say that $\mathbf{P0}$ is a non-convex MINLP. Here we take the inequality in constraint (17) and prove its non-convexity. For the convenience of expression, we define function

$$c(\mu_{v_i}) = \frac{LM}{x_i \mu_{v_i} f_{u_j} + (1 - x_i) f_{v_i}} - \tau, \tag{43}$$

so that constraint (17) can be written as $c(\mu_{v_i}) \leq 0$. Assume the binary viable x_i is known, the first order derivative of $c(\mu_{v_i})$ is calculated as

$$c'(\mu_{v_i}) = \frac{-LMx_i\mu_{v_i} f_{u_j}}{\left[x_i \mu_{v_i} f_{u_j} + (1 - x_i) f_{v_i}\right]^2}, \tag{44}$$

and the second derivative is

$$C''(\mu_{v_i}) = LMx_i \left[x_i \mu_{v_i} f_{u_j} + (1 - x_i) f_{v_i}\right] \times \frac{2x_i f_{u_j}^2 - x_i \mu_{v_i} f_{u_j} + (x_i - 1) f_{v_i}}{\left[x_i \mu_{v_i} f_{u_j} + (1 - x_i) f_{v_i}\right]^4}. \tag{45}$$

When x_i is set as 1, we have

$$C''(\mu_{v_i})_{|x_i=1} = \frac{-LM\mu_{v_i} f_{u_j} \left(2f_{u_j}^2 - 1\right)}{\mu_{v_i} f_{u_j}^4}. \tag{46}$$

It can be seen that $C''(\mu_{v_i})_{|x_i=1}$ is less than 0 when $2f_{u_j}^2 - 1 > 0$, i.e., $f_{u_j} > 0.71$. The non-convex property of the inequality constraint (17) is proved, and hence problem $\mathbf{P0}$ is a non-convex MINLP. This completes the proof.

References

[1] S. Feng, X. Yan, H. Sun, Y. Feng, and H. X. Liu, "Intelligent driving intelligence test for autonomous vehicles with naturalistic and adversarial environment," *Nature Communications*, vol. 12, no. 1, p. 748, Feb. 2021.

[2] D. Wang, W. Fu, Q. Song, and J. Zhou, "Potential risk assessment for safe driving of autonomous vehicles under occluded vision," *Scientific Reports*, vol. 12, no. 1, p. 4981, Mar. 2022.

[3] L. Claussmann, M. Revilloud, D. Gruyer, and S. Glaser, "A review of motion planning for highway autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 5, pp. 1826–1848, May 2020.

[4] D. Omeiza, H. Webb, M. Jirotka, and L. Kunze, "Explanations in autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 10 142–10 162, Aug. 2022.

[5] S. Sridhar and A. Eskandarian, "Cooperative perception in autonomous ground vehicles using a mobile-robot testbed," *IET Intelligent Transport Systems*, vol. 13, no. 10, pp. 1545–1556, July 2019.

[6] A. Guizar, V. Mannoni, F. Poli, B. Denis, and V. Berg, "Lte-v2x performance evaluation for cooperative collision avoidance (coca) systems," in *IEEE 92nd Vehicular Technology Conference (VTC)*, Victoria, BC, Canada, Nov. 2020.

[7] Y. Qi, Y. Zhou, Y.-F. Liu, L. Liu, and Z. Pan, "Traffic-aware task offloading based on convergence of communication and sensing in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 8, no. 24, pp. 17 762–17 777, May 2021.

[8] M. Cui, S. Zhong, B. Li, X. Chen, and K. Huang, "Offloading autonomous driving services via edge computing," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 10 535–10 547, Oct. 2020.

[9] H. Du, S. Leng, K. Zhang, and L. Zhou, "Cooperative sensing and task offloading for autonomous platoons," in *GLOBECOM-IEEE Global Communications Conference*, Taipei, Taiwan, China, Jan. 2020.

[10] B. Yang, X. Cao, K. Xiong, C. Yuen, Y. L. Guan, S. Leng, L. Qian, and Z. Han, "Edge intelligence for autonomous driving in 6g wireless system: Design challenges and solutions," *IEEE Wireless Communications*, vol. 28, no. 2, pp. 40–47, Apr. 2021.

[11] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," *IEEE Micro*, vol. 35, no. 6, pp. 60–68, Dec. 2015.

[12] S.-C. Lin, Y. Zhang, C.-H. Hsu, M. Skach, M. E. Haque, L. Tang, and J. Mars, "The architectural implications of autonomous driving: Constraints and acceleration," *SIGPLAN Not.*, vol. 53, no. 2, p. 751–766, Mar. 2018.

[13] Mobileye, "Enabling Autonomous," 2017, http://www.mobileye.com/future-of-mobility/mobileye-enabling-autonomous.

[14] Udacity, "An Open Source Self-Driving Car," 2017, https://www.udacity.com/self-driving-car.

[15] M. Naumann, H. Konigshof, M. Lauer, and C. Stiller, "Safe but not overcautious motion planning under occlusions and limited sensor range," in *IEEE Intelligent Vehicles Symposium (IV)*, Paris, France, Jun. 2019.

[16] M. Schratter, M. Bouton, M. J. Kochenderfer, and D. Watzenig, "Pedestrian collision avoidance system for scenarios with occlusions," in *IEEE Intelligent Vehicles Symposium (IV)*, Paris, France, Jun. 2019.

[17] C. Zhang, F. Steinhauser, G. Hinz, and A. Knoll, "Improved occlusion scenario coverage with a pomdp-based behavior planner for autonomous urban driving," in *IEEE International Intelligent Transportation Systems Conference (ITSC)*, Indianapolis, IN, Sep. 2021.

[18] P. Ghorai, A. Eskandarian, Y.-K. Kim, and G. Mehr, "State estimation and motion prediction of vehicles and vulnerable road users for cooperative autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–20, Apr. 2022.

[19] Y. Hui, X. Ma, Z. Su, N. Cheng, Z. Yin, T. H. Luan, and Y. Chen, "Collaboration as a service: Digital twins enabled collaborative and distributed autonomous driving," *IEEE Internet of Things Journal*, pp. 1–13, Mar. 2022.

[20] Z. Ning, P. Dong, M. Wen, X. Wang, L. Guo, R. Y. K. Kwok, and H. V. Poor, "5g-enabled uav-to-community offloading: Joint trajectory design and task scheduling," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 11, pp. 3306–3320, Jun. 2021.

[21] Q. Liu, T. Han, J. L. Xie, and B. Kim, "Livemap: Real-time dynamic map in automotive edge computing," in *INFOCOM - IEEE Conference on Computer Communications*, Vancouver, BC, Canada, May 2021.

[22] Y. Qi, Y. Zhou, Z. Pan, L. Liu, and J. Shi, "Crowd-sensing assisted vehicular distributed computing for hd map update," in *ICC - IEEE*

International Conference on Communications, Montreal, QC, Canada, Jun. 2021.
[23] B. Yang, X. Cao, X. Li, C. Yuen, and L. Qian, "Lessons learned from accident of autonomous vehicle testing: An edge learning-aided offloading framework," *IEEE Wireless Communications Letters*, vol. 9, no. 8, pp. 1182–1186, Aug. 2020.
[24] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, and W. Shi, "Edge computing for autonomous driving: Opportunities and challenges," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1697–1716, Jun. 2019.
[25] J. Du, F. R. Yu, X. Chu, J. Feng, and G. Lu, "Computation offloading and resource allocation in vehicular networks based on dual-side cost minimization," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1079–1092, 2019.
[26] W. Feng, N. Zhang, S. Li, S. Lin, R. Ning, S. Yang, and Y. Gao, "Latency minimization of reverse offloading in vehicular edge computing," *IEEE Transactions on Vehicular Technology*, vol. 71, no. 5, pp. 5343–5357, May 2022.
[27] M. Liu and Y. Liu, "Price-Based Distributed Offloading for Mobile-Edge Computing With Computation Capacity Constraints," *IEEE Wireless Communications Letters*, vol. 7, no. 3, pp. 420–423, Jun. 2018.
[28] Y. Huang, Y. Liu, and F. Chen, "NOMA-Aided Mobile Edge Computing via User Cooperation," *IEEE Transactions on Communications*, vol. 68, no. 4, pp. 2221–2235, Apr. 2020.
[29] B. Dai, F. Xu, Y. Cao, and Y. Xu, "Hybrid sensing data fusion of cooperative perception for autonomous driving with augmented vehicular reality," *IEEE Systems Journal*, vol. 15, no. 1, pp. 1413–1422, Mar. 2021.
[30] Y. Liu, S. Wang, Q. Zhao, S. Du, A. Zhou, X. Ma, and F. Yang, "Dependency-aware task scheduling in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 4961–4971, 2020.
[31] W. Mao, O. U. Akgul, A. Mehrabi, B. Cho, Y. Xiao, and A. Ylä-Jääski, "Data-Driven Capacity Planning for Vehicular Fog Computing," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13 179–13 194, Aug. 2022.
[32] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, "Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–14, Jan. 2022.
[33] T. S. Etsi, "Systems (its); vehicular communications; basic set of applications; part 2: Specification of cooperative awareness basic service, v1.4.1, document etsi en 302 637- 2, nat. highway traffic safety admin., washington, dc, usa," 2019. [Online]. Available: https://www.etsi.org/deliver/etsi_en/302600_
[34] K. Sung, K. Min, and J. Choi, "Driving information logger with in-vehicle communication for autonomous vehicle research," in *20th International Conference on Advanced Communication Technology (ICACT)*, Chuncheon, Korea (South), Feb. 2018.
[35] Y. Wang, G. de Veciana, T. Shimizu, and H. Lu, "Deployment and performance of infrastructure to assist vehicular collaborative sensing," in *IEEE 87th Vehicular Technology Conference (VTC Spring)*, Porto, Portugal, Jun. 2018.
[36] H. A. Ignatious, Hesham-El-Sayed, and M. Khan, "An overview of sensors in autonomous vehicles," *Procedia Computer Science*, vol. 198, pp. 736–741, Dec. 2021.
[37] H. Hu, Z. Liu, S. Chitlangia, A. Agnihotri, and D. Zhao, "Investigating the impact of multi-lidar placement on object detection for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 2550–2559.
[38] F. Bounini, D. Gingras, V. Lapointe, and H. Pollart, "Autonomous vehicle and real time road lanes detection and tracking," in *IEEE Vehicle Power and Propulsion Conference (VPPC)*, Montreal, QC, Canada, Oct. 2015.
[39] S. Markowitz, C. Snyder, Y. C. Eldar, and M. N. Do, "Multimodal unrolled robust pca for background foreground separation," Aug. 2021.
[40] C. H. Papadimitriou, "On the complexity of integer programming," *J. ACM*, vol. 28, no. 4, p. 765–768, Oct. 1981.
[41] P. Belotti, C. Kirches, S. Leyffer, J. Linderoth, J. Luedtke, and A. Mahajan, "Mixed-integer nonlinear optimization," *Acta Numerica*, vol. 22, pp. 2–3, May 2013.
[42] X.-S. Yang, *Cuckoo Search and Firefly Algorithm: Overview and Analysis*, Cham, 01 2014, vol. 516, pp. 1–26. [Online]. Available: https://doi.org/10.1007/978-3-319-02141-6_1
[43] E. Zadobrischi, M. Dimian, and M. Negru, "The utility of dsrc and v2x in road safety applications and intelligent parking: Similarities, differences, and the future of vehicular communication," *Sensors (Basel, Switzerland)*, vol. 21, no. 21, pp. 1–38, Oct. 2021.
[44] K. Wang, Y. Zhou, Z. Liu, Z. Shao, X. Luo, and Y. Yang, "Online task scheduling and resource allocation for intelligent noma-based industrial internet of things," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 5, pp. 803–815, Mar. 2020.
[45] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for iot applications: A learning-based approach," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1117–1129, Mar. 2019.
[46] Y. Luo, "Time constraints and fault tolerance in autonomous driving systems," Master's thesis, EECS Department, University of California, Berkeley, May 2019.
[47] T. Cai, Z. Yang, Y. Chen, W. Chen, Z. Zheng, Y. Yu, and H.-N. Dai, "Cooperative data sensing and computation offloading in uav-assisted crowdsensing with multi-agent deep reinforcement learning," *IEEE Transactions on Network Science and Engineering*, pp. 1–15, Oct. 2021.
[48] M. Hofbauer, C. B. Kuhn, J. Meng, G. Petrovic, and E. Steinbach, "Multi-view region of interest prediction for autonomous driving using semi-supervised labeling," in *IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, Rhodes, Greece, Sep. 2020.
[49] T.-H. Wang, S. Manivasagam, M. Liang, B. Yang, W. Zeng, and R. Urtasun, "V2vnet: Vehicle-to-vehicle communication for joint perception and prediction," in *Computer Vision – ECCV 2020*, A. Vedaldi, H. Bischof, T. Brox, and J.-M. Frahm, Eds. Cham: Springer International Publishing, Aug. 2020, pp. 605–621.
[50] A. Abbas, A. Raza, F. Aadil, and M. Maqsood, "Meta-heuristic-based offloading task optimization in mobile edge computing," *International Journal of Distributed Sensor Networks*, vol. 17, no. 6, pp. 1–11, Jun. 2021.
[51] Z. Wang and R. Hou, "Joint caching and computing resource allocation for task offloading in vehicular networks," *IET Communications*, vol. 14, no. 21, pp. 3820–3827, Oct. 2020.
[52] W. Zhou, W. Fang, Y. Li, B. Yuan, Y. Li, and T. Wang, "Markov approximation for task offloading and computation scaling in mobile edge computing," *Mobile Information Systems*, vol. 2019, pp. 1–12, Jan. 2019.
[53] K. Xiong, S. Leng, C. Huang, C. Yuen, and Y. L. Guan, "Intelligent task offloading for heterogeneous v2x communications," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 4, pp. 2226–2238, 2021.
[54] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing," *IEEE Transactions on Mobile Computing*, pp. 1–18, Jan. 2022.
[55] O. Zedadra, A. Guerrieri, and H. Seridi, "Lfa: A leacute;vy walk and firefly-based search algorithm: Application to multi-target search and multi-robot foraging," *Big Data and Cognitive Computing*, vol. 6, no. 1, pp. 1–15, Feb. 2022.
[56] H. Ibn-Khedher, M. Laroui, M. B. Mabrouk, H. Moungla, H. Afifi, A. N. Oleari, and A. E. Kamal, "Edge computing assisted autonomous driving using artificial intelligence," in *International Wireless Communications and Mobile Computing (IWCMC)*, Harbin City, China, Jul. 2021, pp. 254–259.
[57] R. Castellano, "Intel's Mobileye: Strong Headwinds From Competitor SoCs, Particularly In China," 2021, https://seekingalpha.com/article/4474916-intel-mobileye-strong-headwinds-china-soc-competitors.

Zhu Xiao (M'12-SM'19) received the M.S. and Ph.D. degrees in communication and information systems from Xidian University, China, in 2007 and 2009, respectively. From 2010 to 2012, he was a Research Fellow with the Department of Computer Science and Technology, University of Bedfordshire, U.K.

He is currently an Associate Professor with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include wireless localization, Internet of Vehicles and intelligent transportation systems. He is a senior member of the IEEE and serving as an Associated Editor for IEEE Transactions on Intelligent Transportation Systems.

Jinmei Shu received her BS degree in communication engineering from Jishou University, Jishou, China, in 2021. She is currently working toward the MS degree in information and communication engineering at Hunan University, Changsha, China. Her current research interests include connected and autonomous vehicles and mobile edge computing.

Hongbo Jiang (Senior Member, IEEE) received the Ph.D. degree from Case Western Reserve University in 2008. He is currently a Full Professor with the College of Computer Science and Electronic Engineering, Hunan University. He ever was a Professor with the Huazhong University of Science and Technology. His research concerns computer networking, especially algorithms and protocols for wireless and mobile networks. He was the Editor of IEEE/ACM TRANSACTIONS ON NETWORKING, the Associate Editor for IEEE TRANSACTIONS ON MOBILE COMPUTING, and the Associate Technical Editor for IEEE Communications Magazine. He is an Elected Member of Academia Europaea, Fellow of IET, Fellow of BCS, and Fellow of AAIA.

Geyong Min is a Professor of High Performance Computing and Networking in the Department of Computer Science within the College of Engineering, Mathematics and Physical Sciences at the University of Exeter, United Kingdom. He received the PhD degree in Computing Science from the University of Glasgow, United Kingdom, in 2003, and the B.Sc. degree in Computer Science from Huazhong University of Science and Technology, China, in 1995. His research interests include Future Internet, Computer Networks, Wireless Communications, Multimedia Systems, Information Security, High Performance Computing, Ubiquitous Computing, Modelling and Performance Engineering.

Hongyang Chen (M'10-SM'16) received the B.S. degree from Southwest Jiaotong University, Chengdu, China, in 2003, the M.S, degree from the Institute of Mobile Communications, Southwest Jiaotong University, Chengdu, China, in 2006, and the Ph.D. degree from the University of Tokyo, in 2011. From 2004 to 2006, he was a Research Assistant with the Institute of Computing Technology, Chinese Academy of Science. In 2009, he was a Visiting Researcher with the UCLA Adaptive Systems Laboratory, University of California Los Angeles. From April 2011 to June 2020, He worked as a Researcher with Fujitsu Ltd, Japan.

He is currently a Principal Investigator/Senior Research Scientist with Zhejiang Lab, China. He has authored or coauthored more than 80 referred journal and conference papers. He has granted/filled more than 50 PCT patents. He was the Editor for IEEE TRANSACTION ON WIRELESS COMMUNICATIONS, and the Associate Editor for IEEE COMMUNICATIONS LETTERS. His research interests include data-driven intelligent networking and system, machine learning, localization and location based big data, B5G, statistical signal processing.

Zhu Han (S'01-M'04-SM'09-F'14) received the B.S. degree in electronic engineering from Tsinghua University, in 1997, and the M.S. and Ph.D. degrees in electrical and computer engineering from the University of Maryland, College Park, in 1999 and 2003, respectively.

From 2000 to 2002, he was an R&D Engineer of JDSU, Germantown, Maryland. From 2003 to 2006, he was a Research Associate at the University of Maryland. From 2006 to 2008, he was an assistant professor at Boise State University, Idaho. Currently, he is a John and Rebecca Moores Professor in the Electrical and Computer Engineering Department as well as in the Computer Science Department at the University of Houston, Texas. His research interests include wireless resource allocation and management, wireless communications and networking, game theory, big data analysis, security, and smart grid. Dr. Han received an NSF Career Award in 2010, the Fred W. Ellersick Prize of the IEEE Communication Society in 2011, the EURASIP Best Paper Award for the Journal on Advances in Signal Processing in 2015, IEEE Leonard G. Abraham Prize in the field of Communications Systems (best paper award in IEEE JSAC) in 2016, and several best paper awards in IEEE conferences. Dr. Han was an IEEE Communications Society Distinguished Lecturer from 2015-2018, AAAS fellow since 2019, and ACM distinguished Member since 2019. Dr. Han is a 1% highly cited researcher since 2017 according to Web of Science. Dr. Han is also the winner of the 2021 IEEE Kiyo Tomiyasu Award, for outstanding early to mid-career contributions to technologies holding the promise of innovative applications, with the following citation: "for contributions to game theory and distributed management of autonomous communication networks."